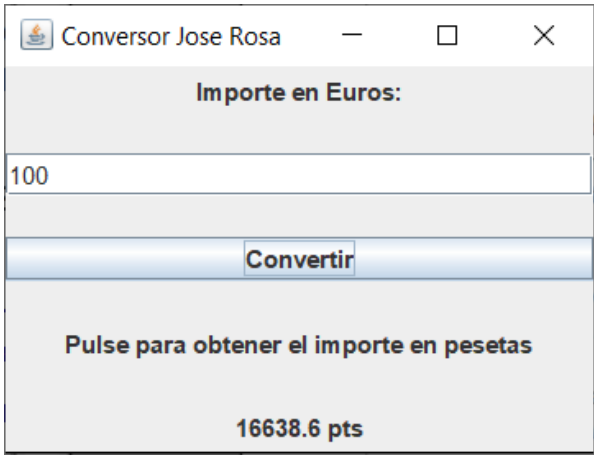
Resuelve los siguientes ejercicios y entrega el código fuente tanto en un fichero .pdf como en ficheros de texto java o la aplicación ejecutable según se pida en el ejercicio, en el .pdf capturas de pantalla (si se piden) y las respuestas a las preguntas que se plantean, si las hay.

EJERCICIOS PADAWAN (OBLIGATORIOS)

EJERCICIO 1 (1 PUNTO). DEFINIR GUI DESDE CÓDIGO.

Realiza un nuevo proyecto llamado **E701**, **construye de forma manual** (no vale usar ningún diseñador de interfaces) una una GUI similar a la de la figura y que nos permita realizar la conversión de Euros a Pesetas, tal y como se muestra en la figura.



- ***Pon tu nombre y apellido en el título de la ventana.***
- *Usa un Layout Manager (en el contentPane de la ventana) de tipo GridLayout y te aseguras que se mantenga la distribución de controles aunque se cambie el tamaño de la ventana.*
- Cuando pulses el botón "Convertir", el manejador del evento comprueba que el texto no sea vacío (en caso de serlo, que no pase nada). Los componentes usados son además del JButton, varios *JLabel* y un *JTextField.*

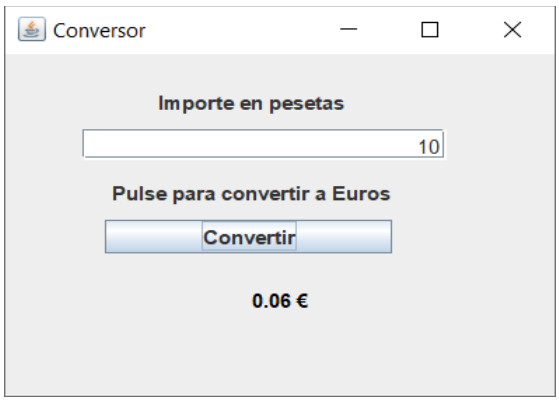
Consejos:

- No te olvides del form, de su content pane, de su layout manager, etc.
- Tampoco olvides crear el listener y registrarlo para el botón.
- Los componentes de la GUI con los que el programa necesite interactuar desde varios lugares, que tengan referencias como variables miembro de la clase (globales).
- La forma de dar nombre a los componentes que vayas a utilizar desde sentencias del programa es importante. Prefija los nombres con lo que son: si son labels, que comiencen por "lb", si son botones que comiencen por "b", etc. Luego debe seguirle lo que hacen o contienen (bConvertir podría ser el nombre de la variable que referencia al botón que convierte), si es un textfield que contiene el importe, "tfImporte", etc. Eso te ayudará a no liarte en el código.
- El nombre de los componentes que no vayas a usar desde el código (solo para visualizarlos y poco más), no importa mucho como se llamen: label1, label2, …

EJERCICIO 2 (1 PUNTO). DEFINIR GUI DESDE DISEÑADOR.

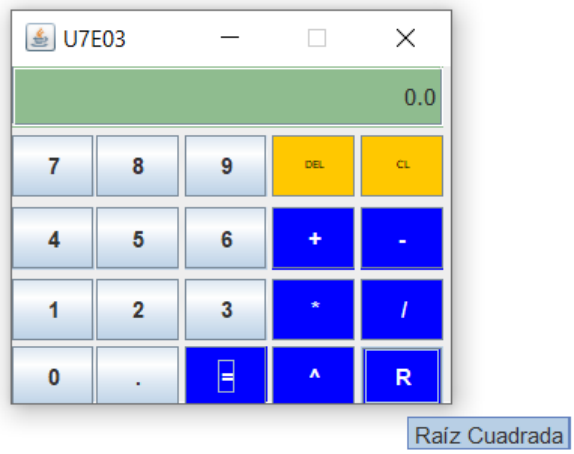
Realiza un nuevo proyecto similar al anterior, pero ahora utiliza WindowBuilder para definir la interfaz. El proyecto se va a llamar U7E02, y va a convertir Euros a Pesetas, de forma similar al anterior pero al contrario. **Pon tu nombre en el título de la ventana**. Además, otras

diferencias que tendrá es que como Layout manager usaremos el absoluto o nulo y añadiremos como funcionalidad el hecho de que si al intentar convertir la cantidad escrita en la caja de texto se produce un error, vamos a imprimir en el label del resultado el mensaje de error y el fondo de la caja de texto y la letra del label resultado en rojo. Si todo va bien, el color del texto del label será negro y el fondo de la caja de edición será blanco.



EJERCICIO 3 (1 PUNTO). UNA CALCULADORA.

Realiza un nuevo proyecto llamado **U7E03** que muestre una interfaz de usuario que nos permita (observando una real podemos copiar su comportamiento) tener la funcionalidad de una calculadora. **Pon tu nombre y primer apellido en el título de la ventana**.



SOBRE LA INTERFAZ:

Por un color distinto a las teclas de limpiar, los dígitos y las operaciones.

Haz que las teclas que no sean dígitos tengan un mensaje de tooltip (pequeña ayuda visible cuando pasas el puntero del ratón por encima) Por ejemplo lo que aparece en el botón [R].

En vez de hacer un listener para cada botón, intenta compartirlos, por ejemplo, en un solo listener la pulsación de los dígitos.

Tendrá las operaciones básicas (suma, resta, multiplicación, división, cuadrado y raíz cuadrada) y botones para borrar el último dígito **[DEL]** y poner a 0 el display **[CL]** o hacer una operación binaria (intervienen dos valores) que tenga la cadena **[=]** o unarias (operación con un valor) como el cuadrado y la raíz cuadrada. Debes:

- Impedir que el usuario escriba en el display (busca si tiene una propiedad solo lectura).
- Al pulsar un botón dígito o punto decimal → si el display contiene "Error" dejar en el display el dígito pulsado. Si es diferente, añadir al display (la caja de texto) por el final, la cifra pulsada. Si es el punto, añadirlo solamente si no hay otro punto antes.
- Pulsar [DEL] → borrar al display la última letra (si la hay).

- Pulsar operación binaria (suma, resta, multiplicar y dividir) → usar como primer operando el valor del display, borrar el display y esperar el segundo operando y la pulsación de la tecla [=] u otra operación. Si al pulsar en una operación ya había una operación binaria anterior pendiente y se tiene el segundo valor, realiza la operación. Si no se tiene el segundo valor, sustituir la operación anterior por la nueva.
- Pulsar operación unaria (cuadrado y raíz cuadrada): aplicar la operación al valor del display y cambiarlo por su resultado. Si es la raíz cuadrada de un número negativo, mostrar en el display "Error".
  Pulsar igual [=]: mostrar el resultado de una operación si está pendiente o no hacer nada. Si es una división por 0, mostrar "Error" en el display.

EJERCICIO 4 (1 PUNTO). UN PRIMER JUEGO DE TIPO ARCADE.

Este ejercicio es una especie de tutorial guiado en el que tendrás que ir realizando una serie etapas (etapa1, etapa2, ..) divididas en pasos. Crea un package para cada etapa y cuando finalices una etapa, copia todo lo que tengas dentro del package de la siguiente (así no perderás lo que tienes conseguido), es una forma cutre de gestión de versiones.

ETAPA 1. LA ESTRUCTURA DEL JUEGO.

Comenzamos el desarrollo usando un editor de texto como notepad++. Crea la carpeta llamada "bichos" y dentro de ella otra llamada "etapa1". Ahora sigue los siguientes pasos.

PASO 1.1: DEFINIR LA VENTANA DE LA APLICACIÓN.

El programa crea una ventana Swing (**JFrame**) y arbitrariamente decidimos que el tamaño de la ventana será de 800 x 600 píxels. Declaramos estas dimensiones como constantes para que modificarlas más tarde sea trivial y no nos obligue a cambiar el programa. Usa este código y pon tu nombre y primer apellido al título de la ventana.

```java
package etapa1;

import javax.swing.JFrame;

public class Bichos  {
    public static final int ANCHO = 800;
    public static final int ALTO = 600;

    public Bichos() {
        JFrame ventana = new JFrame("Bichos");
        ventana.setBounds(0,0, ANCHO, ALTO);
        ventana.setVisible(true);
    }

    public static void main(String[] args) {
        Bichos b = new Bichos();
    }
}
```

Para conseguir que se cierre la aplicación al pulsar el aspa de la barra de título, vamos a utilizar un método diferente de como lo hemos realizado hasta ahora. Añadimos un adaptador de **java.awt.event**. Añade este código al final del constructor de la ventana:

```
ventana.addWindowListener(
    new WindowAdapter() { public void windowClosing(WindowEvent e) { System.exit(0);} }
);
```

PASO 1.2: NECESITAMOS DIBUJAR/PINTAR COSAS EN LA VENTANA.

Esta definición de la clase Bichos nos genera un problema: para pintar necesitamos sobrescribir el método **paint()** de "alguien" -de alguna ventana, o de algún panel. Sin embargo, ni la clase Bichos es una ventana ni tampoco podríamos hacer nada si lo fuese, ya que la ventana que se muestra es otra que creamos dentro del constructor de Bichos.

Una **posible solución es hacer descender nuestra clase Bichos de alguna que tenga el método paint(), por ejemplo la clase Canvas** y siendo un Canvas, la clase Bichos hereda el su método paint() que podemos sobreescribir. ¿Cómo hacer la asociación entre nuestra clase y la ventana? siendo **Canvas** un descendiente de **Component**, podemos añadir al **contentPane()** de la ventana que crea Bichos, al propio objeto Bichos como un componente más.

```
// Añade al código de la clase Bichos
import javax.swing.JPanel;

import java.awt.Canvas;
import java.awt.Dimension;
import java.awt.Color;

// Extiende la clase Bichos de un Canvas
public class Bichos extends Canvas { ... }

// Añade sentencias en el constructor de Bichos, tras crear la ventana
// quita el layout del panel y añade al content pane de la ventana el objeto Bichos
JPanel panel = (JPanel)ventana.getContentPane();
panel.setBounds(0, 0, ANCHO, ALTO);  // Redimensionar el panel y la ventana
ventana.setBounds(0, 0, ANCHO, ALTO);
panel.setPreferredSize( new Dimension(ANCHO, ALTO) );
panel.setLayout( null );   // Anular el Layout Manager
panel.add( this );         // Añadir el objeto Bichos como componente de la ventana
```

Añade a la clase Bichos el método:

```
public void paint( Graphics g ) {
    g.setColor( Color.red );
    g.fillOval( ANCHO/2 - 10, ALTO/2 - 10, 20, 20);
}
```

Prueba el código a ver si funciona. **Pasa una captura de la ventana** (pulsa teclas ALT+Impr.Pant) la copias al Paint, recortas y guardas como **"bichos12.png"**.

PASO 1.3: NECESITAMOS DIBUJAR IMÁGENES.

Para dar al juego un aspecto algo vistoso usamos imágenes externas que iremos cargando según se necesiten. Hay que cargar una imagen a partir de un fichero y mostrarla sobre la ventana. Para tener transparencia vamos a utilizar imágenes en formato .gif (.png también admite transparencia, sin embargo .jpg no). Crea la carpeta "res" (de recursos) dentro de "etapa1" y copia en ella la imagen "extra.gif"

Para cargar cada imagen utilizamos la clase **ImageIO**. En lugar de poner una ruta de archivo absoluta, utilizamos llamadas a los métodos:

getClass().getClassLoader().getResource(...)

Lo que permite obtener una URL apuntando a un subdirectorio relativo al sitio del cual fue cargada la clase. Esto permite que nuestro programa funcione de forma transparente si decidimos convertirlo en programa ejecutable.

```
// Añade estas sentencias import
// Sería más útil si intentas agruparlas por prefijo (todos los java.wt juntos...)
import javax.imageio.ImageIO;

import java.awt.event.*;
import java.awt.Graphics;
import java.awt.image.BufferedImage;

import java.net.URL;
```

Definimos el método **cargaImagen()** en la clase Bichos:

```
public BufferedImage cargaImagen( String nombre ) {
    URL url = null;
    try {
        url = getClass().getClassLoader().getResource(nombre);
        return ImageIO.read( url );
    } catch ( Exception e ) {
        System.out.println("Error al cargar imagen " + nombre + " de " + url );
        System.out.println("ERROR: " + e.getClass().getName() + " " + e.getMessage() );
        System.exit(0);
        return null;
    }
}
```

Cambiamos el código dentro del método **paint()** por este otro para ver si se dibuja:

```
BufferedImage extra = cargaImagen("res/extra.gif");
g.drawImage( extra, 40, 40, this );
```

Ejecuta el programa y **pasa una captura de la ventana** (pulsa teclas ALT + Impr.Pant) la copias al Paint, recortas y guardas como **"Bichos13.png"**.

PASO 1.4: MEJORAR LA CARGA DE IMÁGENES.

La forma de cargar la imagen es patética. Al hacerlo dentro de ***paint()***, la cargamos ***cada vez que se redibuja la ventana***, algo no muy hábil. Se podría pensar que lo ideal es cargar todas las imágenes/recursos de una aplicación en su constructor, pero pensemos por un momento qué ocurriría si:

- Usamos miles de recursos y con mucho peso (imágenes de distintos bichos y niveles, sonidos...).
- Nuestro juego se ejecuta en un navegador...

Al descargar el usuario las clases Java del juego, se quedaría esperando una eternidad a que el programa descargase todos los recursos, que realmente, igual no llega ni a utilizar porque el jugador/usuario del programa no alcanza el nivel en el que se necesitan/no necesita todos.

Por este motivo se suele utilizar una técnica llamada ***carga tardía***, que consiste en cargar la imagen/recurso una única vez, pero aplazar ese momento hasta que realmente sea necesario. Sacamos esto a nivel de clase:

```
public BufferedImage extra = null;
```

Y en paint(), cargamos solo si es necesario:

```
if( extra == null) extra = cargaImagen("res/extra.gif" ); // En windows '/' tb. Funciona!
```

Hemos mejorado, pero es un engorro tener que estar comprobando todo el rato en todas las posibles situaciones donde nos pueda surgir la necesidad de utilizar una imagen si está cargada o no. Para evitarlo, vamos a encapsular esto en una rutina a la que llamaremos ***getSprite()***, que recibirá como parámetro un nombre de sprite (personaje del juego) que necesitemos y devolverá su imagen, cargándola si es necesario. Puesto que puede haber muchas imágenes, la rutina tendrá que recordar qué sprites están cargados y cuáles no. Para ello nada mejor que un ***HashMap*** que asocie el nombre de cada sprite con su imagen en memoria.

> **Nota:** *una tabla hash (eso es HashMap) es una colección de parejas (clave, valor) que es capaz de localizar rápidamente un valor usando su clave. Algunos lenguajes los llaman* ***arrays asociativos****, porque son como un array, pero la clave no tiene porqué ser una posición entera (como en los arrays), sino que puede ser un nombre. También podrías pensar en un array como una tabla hash donde las claves son números enteros.*

Importar la clase, declarar en la clase Bichos un HashMap llamado sprites y crearlo en el constructor de Bichos:

```
import java.util.HashMap;

// En la clase Bichos se declara
public HashMap sprites;

// En el constructor de Bichos se crea
sprites = new HashMap();
```

Define el método:

```
public BufferedImage getSprite( String nombre ) {
    BufferedImage img = (BufferedImage)sprites.get(nombre);
    if( img == null ) {
        img = cargaImagen(nombre);
        sprites.put(nombre, img);  // Añade la pareja (clave, valor) a la tabla hash
    }
    return img;
}
```

Y comenta o borra el código que había en el método paint() y lo sustituyes por este:

```
g.drawImage( getSprite("res/extra.gif"), 40, 40, this );
```

PASO 1.5. EL BUCLE PRINCIPAL DEL JUEGO.

Ahora que ya podemos cargar imágenes de forma cómoda, es hora de plantearnos que las cosas cambien en nuestro juego. De la misma forma en que todas las aplicaciones de Windows tienen un núcleo llamado "bucle de proceso de mensajes" o *Message Loop*, algo muy similar sucede con los juegos. Un juego básicamente repite lo siguiente:

1. Actualizar el "estado de su mundo" o "escenario".
2. *Si es preciso*, refresca la pantalla.
3. Volver al punto 1

En el paso 1 es donde ocurren todos los movimientos de personajes, las acciones del jugador, la aparición o desaparición de elementos, los cambios de nivel, la comprobación de si se ha terminado el juego, etc. Por supuesto todo esto no se hace en la misma rutina, pero sí se puede pensar que constituye una funcionalidad claramente diferenciada de la siguiente, en la que si ha habido cambios en lo que el jugador debe ver, se dibuja de nuevo el mundo.

El bucle principal estará en un método de Bichos llamado **juego()**. El método que actualiza el estado del mundo será **actualizaMundo()** y el que dibuja la pantalla de momento seguirá siendo **paint()**.

En este paso, la "actualización del mundo" consiste en que el extraterrestre cambie de posición de forma aleatoria. Por eso necesitamos dos variables globales llamadas posX y posY que guarden su posición, de forma que el método **actualizaMundo()** pueda modificarlas y el método **paint()** pueda utilizarlas para pintar al personaje en su nueva posición. Añade los métodos y cambia el código de **paint()** a:

```
public void actualizaMundo() {
    // posX y posY debes declararlas globales y private
    posX = (int)( Math.random() * ANCHO );
    posY = (int)( Math.random() * ALTO );
}

public void juego() {
    while( isVisible() ) { // es visible el canvas
       actualizaMundo();
       paint( getGraphics() );
    }
}
```

```
g.drawImage( getSprite("res/extra.gif"), posX, posY, this );
```

Tras crear el objeto Bichos en el main() haz una llamada a su método juego():

```
b.juego();
```

Además haz que la ventana no pueda cambiar de tamaño y da un valor inicial a posX y a posY para que el extraterrestre esté inicialmente en el centro de la ventana (ANCHO/2, ALTO/2).

Ejecuta el programa y **pasa una captura de la ventana de la aplicación** (pulsa teclas ALT + Impr.Pant) la copias al Paint, recortas y guardas como **"Bichos15.png"**.

PASO 1.6: EVITAR EL COLAPSO DEL ESCENARIO.

Los cambios anteriores nos deja una invasión de bichos que habría que solucionar. Para ello, vamos a probar inicialmente la estrategia de borrar a mano toda la ventana *antes* de pintar el alien. El único cambio consiste en las siguientes líneas dentro de paint() y antes de dibujar el alien (ejecuta y sigue leyendo):

```
g.setColor( getBackground() );
g.fillRect(0, 0, getWidth(), getHeight());
```

Si ejecutas ahora el programa, verás que el alien cambia de posición tan rápido que apenas da tiempo a verlo o directamente no lo ves. Está claro que hay que insertar artificialmente una pausa en el bucle del juego para que el jugador tenga tiempo de ver algo en pantalla:

1. Actualizar el "estado del mundo" o el "escenario"
2. *Si es preciso*, refrescar la pantalla.
3. Esperar un intervalo de tiempo para ralentizar el proceso.
4. Volver al punto 1

El intervalo de tiempo al que llamamos **RETARDO** será una constante entera que se define a nivel global e inicialmente se fija a 10 (milisegundos). Para ralentizar, en el método juego(), tras dibujar la pantalla y dentro del bucle escribe el código.

```
try {
    Thread.sleep( RETARDO );
} catch ( InterruptedException e ) {}
```

Prueba con diferentes valores del retardo hasta que consigas verle la cara al extraterrestre.
¿Qué valor has dejado al final? _____

Aunque no lo parece, la forma de limpiar tiene un problema. Vamos a hacer que el alien se mueva de forma horizontal y que "rebote" contra los límites de la ventana. Habitualmente este tipo de movimientos se implementan definiendo una variable vX que servirá como "velocidad" e indicará la cantidad de distancia que hay que sumar en cada "ciclo" (cada iteración del bucle del juego) a la variable posX que controla su posición horizontal.

Si la variable que representa la velocidad es positiva (vX > 0), entonces la posición X irá aumentando y por lo tanto el alien se moverá a la derecha. Si la variable vX es negativa, la posición X irá disminuyendo y el alien se moverá a la izquierda.

La forma de controlar los rebotes es comprobando la posición después de cada movimiento. Si se sale por la izquierda (posX < 0) o por la derecha (posX > ANCHO - 10), entonces hay que cambiar la dirección del movimiento. Pero tal como lo tenemos planteado, cambiar la dirección del movimiento es simplemente cambiar el signo de la velocidad. Si antes de rebotar vX valía 5, después de rebotar pasará a valer -5 y viceversa. Multiplicando por -1 lo conseguimos.

Declara la variable miembro vX de tipo entero y dale el valor 2. Y en actualizaMundo() añade:

```
posX += vX;
if( posX < 0 || posX > ANCHO-10 ) vX = -vX;
```

Prueba el código y ajusta el valor de RETARDO y de vX para que el alien se desplace de forma suave pero no eterna por la ventana. **¿Valor de RETARDO? _____ ¿Valor de vX? _____**

ETAPA 2. VIGILAR EL RENDIMIENTO.

Crea una nueva carpeta llamada "etapa2" y copia todo lo que tenías de la anterior. Continuamos desarrollando el juego en esta nueva copia para no perdernos entre tantos pasos.

PASO 2.1. IMPLEMENTAR DOBLE-BUFFERING DE FORMA MANUAL.

Salvo que tengas un ordenador excepcionalmente rápido, observarás que el programa tiene un defecto muy molesto y es un parpadeo en el alien o que va a trompicones. Ocurre porque durante unos instantes el ojo está viendo la pantalla recién borrada en la que todavía no se ha pintado ningún alien. Si el ordenador es rápido y no podemos apreciarlo, basta con aumentar el tamaño de la ventana o utilizar gráficos más grandes para que el efecto se aprecie.

Para resolver esta situación utilizamos una técnica llamada **double buffering**. Consiste en tener una imagen en memoria del mismo tamaño que el escenario. En la imagen de RAM se hacen todas las operaciones de dibujo. Una vez que el escenario ha sido pintado "fuera de la pantalla" (off-screen), esa imagen se vuelca encima de la ventana (lo que se ve). A la imagen en memoria se la suele llamar buffer. Así que declara el buffer en la clase Bicho:

```
public BufferedImage buffer;
```

Lo creas en el constructor de la clase:

```
buffer = new BufferedImage(ANCHO, ALTO, BufferedImage.TYPE_INT_RGB );
```

Creamos un nuevo método al que llamamos **pintaMundo()** donde dejamos sentencias parecidas a las que antes teníamos en paint().

```
public void pintaMundo() {
     Graphics g = buffer.getGraphics();  // Un gráfico para dibujar en RAM
     g.setColor( getBackground());       // Hacemos lo mismo
     g.fillRect(0, 0, getWidth(), getHeight() );
     g.drawImage( getSprite("res/extra.gif"), posX, posY, this);
}

public void paint( Graphics g ) {
    g.drawImage( buffer, 0, 0, this);
}
```

El motivo es que ahora en paint() lo único que haremos es copiar el buffer de RAM en la imagen, mientras que dibujar el mundo lo haremos en el buffer.

Por último en el método **juego()**, primero actualizamos el mundo, luego lo pintamos en RAM llamando a **pintaMundo()** y por último se llama a paint().

```
pintaMundo();              // off-screen
paint( getGraphics() );
```

Ejecuta el programa con los cambios a ver si hemos eliminado el parpadeo del alien. ¿Se mueve mejor? ( )Sí ( )No.

La técnica del doble buffer es tan usada que en el JDK 1.4 se incluyó un mecanismo

automático, permitiendo que la visualización del buffer se haga mediante mecanismos más acelerados dependientes de la plataforma (por ejemplo, mover el puntero de memoria de la tarjeta gráfica que indica dónde comienza el buffer de vídeo en lugar de copiar ese buffer).

PASO 2.2. IMPLEMENTAR DOBLE-BUFFERING CON SOPORTE DE JAVA.

La clase que permite implementar este tipo de operaciones se llama **BufferStrategy** y se encuentra en el paquete **java.awt.image**. La forma de utilizarla es extremadamente sencilla:

1. Primero se decide qué ventana/componente es la que tendrá este comportamiento. Se escoge el componente *sobre el que vamos a pintar* (en nuestro caso la clase Bicho).
2. Se llama al método **createBufferStratgy(n)** de ese componente, pasando como parámetro el número de buffers que queremos tener.
3. Utilizamos el método **getBufferStrategy()** del componente para obtener un objeto de tipo **BufferStrategy**, que representa básicamente una lista de buffers de memoria, tantos como valor indica el parámetro que pasamos a createBufferStrategy().
4. En todo momento hay un "buffer de dibujo" (off-screen) y un "buffer visible". Para poder pintar encima del buffer de dibujo, llamamos al método **getDrawGraphics()** del objeto BufferStrategy obtenido anteriormente.
5. Cuando hemos terminado de pintar, llamamos al método **show()** del objeto BufferStrategy para indicar que el buffer está listo y que queremos mostrar el siguiente buffer.

Importa:

```
import java.awt.image.BufferStrategy;
```

Define una variable miembro de la clase Bichos:

```
public BufferStrategy strategy;
```

Al final del constructor de Bichos, antes de hacerla visible añade este código:

```
ventana.setResizable(false);
createBufferStrategy(2);
strategy = getBufferStrategy();
requestFocus();
```

En **pintaMundo()** vamos a cambiar nuestra forma artesana de hacer doble buffer por estas sentencias:

```
Graphics g = strategy.getDrawGraphics();
g.setColor( Color.black );
g.fillRect( 0, 0, getWidth(), getHeight() );
g.drawImage( getSprite("res/extra.gif"), posX, posY, this);
strategy.show();   // Lo dejamos como el buffer activo
```

Y por último comenta el método paint() y la llamada que hacemos a él en el método juego(). Deja RETARDO a 10 y vX entre 2 y 5. Prueba la aplicación a ver como funciona.

PASO 2.3: MEDIR EL RENDIMIENTO PARA SABER QUE COSAS PODEMOS HACER.

Uno de los aspectos fundamentales de cualquier juego es la velocidad con la que es capaz de dibujar el escenario. De nada sirve que el juego sea maravilloso si tardas 2 segundos en dibujarlo. Es muy fácil al principio dejarse llevar por el entusiasmo y comenzar a añadir

opciones y más opciones, con gráficos cada vez más complicados y grandes y al final encontrarse con que redibujar el escenario es tan lento que no es agradable jugar.

Es importante implementar en nuestro juego un contador de "**fotogramas por segundo**" (**fps**) que nos diga cuántas veces por segundo es capaz el programa de dibujar el escenario.

Tampoco tenemos que obsesionarnos con el valor de este contador. Visualmente, un juego a 90 fps es exactamente igual de suave que uno de 1200 fps. El motivo es muy sencillo: si el monitor es de 100 Hz, significa que en un segundo hace 100 barridos de pantalla. Da igual si durante 1/100 segundos de tiempo nuestro programa ha actualizado el escenario mil veces, porque al monitor sólo le da tiempo a dibujar 1 de ellas.

Hay monitores que funcionan a 50, 60 y 75 Hz y en cualquier caso al ojo humano le es suficiente con 50 fps, con lo cual si nuestro contador no baja de esa cifra, no debemos preocuparnos en exceso. Sin embargo, debemos ser conscientes de que los fps limitan las ampliaciones que hagamos en el futuro. Si queremos mil aliens moviéndose en la pantalla y eso nos obliga a llegar a 77 fps, es poco probable que consigamos añadir texturas desplazándose en el fondo sin caer por debajo de 50 fps (umbral de usabilidad).

¿Cómo calcular los fps? Simplemente vamos al bucle principal del juego y medimos cuanto tiempo tarda en realizarse un ciclo con todos sus cambios, actualizaciones, dibujos, etc. Imagina que obtenemos cierta cantidad **m** de milisegundos. Ahora dividimos 1000 (1 segundo) entre m (la cantidad calculada) y obtenemos los fps. Por ejemplo, si el cuerpo de nuestro bucle tarda 14 ms (sin contar con la pausa, que es totalmente artificial y se puede reducir o incluso eliminar), entonces obtenemos 71 fps.

Añade un contador de fps y lo muestras en la esquina inferior izquierda de la pantalla en la posición (20, ALTO – 20) en color verde. Ten en cuenta, que ahora mismo un ciclo es tan rápido que posiblemente tarde menos de 1 ms y por tanto nos dirá que tarda 0 ms, en ese caso, al calcular fps harás una división por 0, generarás una excepción y el programa acabará. Para evitarlo, antes de calcular y mostrar los fps si el tiempo es 0 muestra la cadena "--- fps" y en otro caso "<nº> fps".

Declara la variable fps de tipo int. Mide el tiempo que tarda en actualizar y pintar el mundo. Si el tiempo es distinto de 0, calcula fps, sino deja fps a -1 y así lo detectas en pintaMundo().

```
// Define la variable de instancia
private int fps;     // contador de fotogramas

public void juego() {
    while( isVisible() ) {
        long tiempo = System.currentTimeMillis();
        actualizaMundo();
        pintaMundo();   // off-screen
        tiempo = System.currentTimeMillis() - tiempo;
        if( tiempo != 0 ) fps = (int)(1000L/tiempo); else fps = -1;
        // Retardo artificial
        try { Thread.sleep( RETARDO ); } catch ( InterruptedException e ) {}
    }
}
```

En pintaMundo() pon color verde (Color.green) si fps es menor de 0 muestra la cadena "---fps"

y si es mayor o igual a 0 lo imprimes en la posición.

Muestra una captura de la ventana con el nombre "Bichos23.png".

ETAPA 3. REORGANIZAR EL CÓDIGO APLICANDO POO.

Ya que hemos disfrutado de estudiar la POO, vamos a aplicarla a este código para organizarlo un poco mejor. Inicialmente vamos a pensar que "cosas" (entidades) distintas tenemos en el programa. **Sería conveniente pasarse a un IDE como Eclipse** para no tener problemas de compilación, etc. Crea el proyecto Bichos en Eclipse, un nuevo package bajo src llamado "etapa3". Dentro crea la carpeta "res" y dentro copia la imagen "extra.gif".

> **Nota**: *si prefieres no usar un IDE quizás tengas problemas al generar los .class e incluso al ejecutar. Compilar los fuentes usados desde otros e indícalos a la vez:*
>
> `javac SpriteCache.java Escenario.java Personaje.java Alien.java ...`

PASO 3.1: LA CLASE SPRITECACHE

Tenemos una parte del programa relativamente independiente de las demás, y es el código que se dedica a gestionar la "caché de sprites". Esa parte, que tiene unos datos propios (los sprites cargados) y comportamientos propios (leer y proporcionar sprites) es una típica candidata a convertirse en una clase independiente. Cuando se define una clase nueva, es buena idea pensar qué responsabilidades o interacciones tendrá con el resto. En este caso la clase es muy sencilla y su única interacción es "dar gráficos" a quien los necesite. Por lo tanto su único método público será el siguiente:

```
public BufferedImage getSprite(String nombre) { ... }
```

Una vez tenemos claros los servicios que da la clase, hay que preguntarse qué necesita internamente para llevarlos a cabo. En este caso, necesita el mapa de sprites y el método para cargar una imagen en memoria. La clase **SpriteCache** queda así:

```
package etapa3;

import java.awt.image.BufferedImage;
import java.net.URL;
import java.util.HashMap;
import java.io.File;

import javax.imageio.ImageIO;

public class SpriteCache {
    private HashMap sprites;

    public SpriteCache() { sprites = new HashMap(); }

    private BufferedImage cargaImagen( String nombre ) {
        URL url = null;   // Primero intentamos con url (falla en Eclipse)
        File f = null;    // Intentamos con fichero (falla fuera de Eclipse)
        try {
            url = getClass().getClassLoader().getResource( "etapa3/res/" + nombre );
            return ImageIO.read(url);
        } catch (Exception e) {
            System.out.println("Error al cargar imagen " + nombre + " de " + url);
            System.out.println("ERROR: " + e.getClass().getName() + " " + e.getMessage());
            try {
                url = getClass().getClassLoader().getResource( "etapa3/res/" );
                return ImageIO.read( new File(url.getPath() + nombre ) );
            } catch( Exception e1 ) {
                System.out.println("Error al cargar imagen de fichero " + url);
                System.out.println("ERROR: " + e1.getClass().getName() + " " +
                                   e.getMessage());
                System.exit(0);
                return null;
        }
    }
```

```
    public BufferedImage getSprite(String nombre) {
        BufferedImage img = (BufferedImage)sprites.get(nombre);
        if( img == null ) {
          img = cargaImagen(nombre);
          sprites.put( nombre, img );    // Añadir el sprite al hashMap
        }
        return img;
    }

}  // fin clase SpriteCache
```

PASO 3.2. LA INTERFACE ESCENARIO.

En segundo lugar tenemos el sitio donde se desarrolla la acción. Vamos a llamar a este sitio "el escenario". El escenario es el que de alguna forma "coordina" las cosas que ocurren en el juego porque sabe cuántos aliens hay, cuantas balas, en qué nivel estamos, etc. Una característica muy importante del escenario es que tiene la referencia al único objeto SpriteCache del programa.

Podríamos pensar que ya tenemos una clase "Escenario" que es la propia Bichos.java, pero lo cierto es que si queremos tener funcionalidades que nos sirvan para otros juegos posteriores, será mejor separarla en una clase o una interfaz distinta. Creamos una interfaz con las características que debe tener un escenario.

```
package etapa3;

import java.awt.image.ImageObserver;

public interface Escenario extends ImageObserver {
    public static final int ANCHO = 640;
    public static final int ALTO = 480;
    public static final int RETARDO = 10;

    public SpriteCache getSpriteCache();
}
```

Como se puede ver, de momento esta interfaz lo único que contiene son las constantes generales del juego y un método para acceder al caché de sprites.

PASO 3.4: LA CLASE PERSONAJE Y SUS DESCENDIENTES.

Además tenemos al alien. En el futuro vamos a añadir más personajes y todos serán en esencia lo mismo, si acaso en distintas posiciones o distintas velocidades. Pensemos por un momento qué características comunes van a tener nuestros personajes en el juego para poder definir su clase. Por un lado, todos van a tener:

- Una posición sobre la pantalla.
- Un gráfico que periódicamente se va a pintar. Cada uno tendrá un tamaño que no será necesariamente el mismo para todos.
- Por último, todos van a "hacer algo". Puede que se muevan, disparen, desaparezcan, se teletransporten, ataquen al jugador o cualquier cosa que se nos ocurra. Como ahora mismo no sabemos qué va a ser esa acción, vamos a definir un método genérico llamado **public actua() { ... }** en el que cada personaje elegirá lo que quiere hacer. Lo bueno de este sistema es que el programa principal se limitará a llamar a **personaje.actua()** y

cada uno actuará según su comportamiento, sin que el programa principal tenga que saber nada al respecto.

Estas características las tienen no solo los aliens sino el propio jugador o los disparos que haga. Por lo tanto hemos llegado a una clase que es capaz de encapsular cualquier cosa "activa" sobre el escenario y a esta clase la vamos a llamar Personaje y quedaría como sigue:

```
package etapa3;

import java.awt.Graphics2D;
import java.awt.image.BufferedImage;

public class Personaje {
    protected int x, y;
    protected int ancho, alto;
    protected String spriteNombre;
    protected Escenario escena;
    protected SpriteCache sc;

    public Personaje( Escenario e ) {
        this.escena = e;
        sc = e.getSpriteCache();
    }

    public void paint( Graphics2D g ){
        g.drawImage( sc.getSprite(spriteNombre), x, y, escena );
    }

    public int getX() { return x; }
    public void setX(int i) { x = i; }

    public int getY() { return y; }
    public void setY(int i) { y = i; }

    public String getSpriteNombre() { return spriteNombre; }
    public void setSpriteNombre(String s) {
        spriteNombre = s;
        BufferedImage imagen = sc.getSprite(spriteNombre);
        alto = imagen.getHeight();
        ancho = imagen.getWidth();
    }

    public int getAlto() { return alto; }
    public int getAncho() { return ancho; }
    public void setAlto(int i) { alto = i; }
    public void setAncho(int i) { ancho = i;  }
    public void actua() { }
}
```

```
package etapa3;

public class Alien extends Personaje {
    protected int vx;

    public Alien(Escenario e) {
        super(e);
        setSpriteNombre( "extra.gif" );
    }

    public void actua() {
        x += vx;
        if( x < 0 || x > Escenario.ANCHO - 10 ) vx = -vx;
    }

    public int getVx() { return vx; }
    public void setVx( int i ) { vx = i; }
}
```

Esta es la clase genérica (podríamos haberla hecho abstracta). Ahora, utilizando la herencia, vamos a crear descendientes, definiendo el único personaje que estamos usando hasta ahora, el alien.

PASO 3.5: REESCRIBIR LA CLASE BICHOS.

Toca reescribir la clase original Bichos y aprovechar para añadir la posibilidad de que existan muchos aliens. Todos los almacenamos en una lista llamada "personajes". Observa especialmente los métodos **iniciaMundo()**, **actualizaMundo()** y **dibujaMundo()**.

El primero es nuevo y su propósito es inicializar el escenario. En este caso, se construyen distintos aliens y se añaden a la lista de personajes existentes. Los métodos actualizaMundo() y pintaMundo() hacen lo mismo que antes, pero ahora están más limpios y son independientes de los tipos de personajes con los que trabajan. Cada uno de estos métodos se limita a recorrer la lista de personajes diciéndoles "píntate!", "actúa", etc... y cada personaje concreto, dependiendo de cómo sobreescriba sus métodos, actúa de forma distinta al resto.

```
package etapa3;

import java.awt.Canvas;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Graphics2D;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.awt.image.BufferStrategy;
import java.util.ArrayList;  // Para la lista de personajes

import javax.swing.JFrame;
import javax.swing.JPanel;
```

```
public class Bichos extends Canvas implements Escenario {
    private BufferStrategy strategy;
    private long tiempo;
    private int fps;

    private SpriteCache sc;
    private ArrayList personajes;
```

```
public void iniciaMundo() {
    personajes = new ArrayList();
    for(int i = 0; i < 10; i++) {
        Alien a = new Alien(this);
        a.setX( (int)(Math.random() * Escenario.ANCHO) );
        a.setY( i * 20 );
        int v = (int)(Math.random() * 20 - 10);
        a.setVx( (v == 0)? 1:v );
        personajes.add(a);
    }
}
```

```
public void actualizaMundo() {
    for(int i = 0; i < personajes.size(); i++) {
        Personaje p = (Personaje)personajes.get(i);
        p.actua();
    }
}
```

```
public void pintaMundo() {
    Graphics2D g = (Graphics2D)strategy.getDrawGraphics();
    g.setColor(Color.black);
    g.fillRect(0, 0, getWidth(), getHeight());
    for(int i = 0; i < personajes.size(); i++) {
        Personaje p = (Personaje)personajes.get(i);
        p.paint(g);
    }
    g.setColor(Color.green);
    if( fps > 0 )
        g.drawString( String.valueOf(fps)+ " fps", 10, Escenario.ALTO - 40);
    else
        g.drawString("---- fps", 10, Escenario.ALTO - 40);
    strategy.show();
}
```

```
public SpriteCache getSpriteCache() { return sc; }
```

```
public void juego() {
    fps = 1000;
    iniciaMundo();
    while( isVisible() ) {
        long tiempo = System.currentTimeMillis();
        actualizaMundo();
        pintaMundo();
        tiempo = System.currentTimeMillis() - tiempo;
        if( tiempo > 0 ) fps = (int)(1000/tiempo); else fps = -1;
        try {
            Thread.sleep(RETARDO);
        } catch (InterruptedException e) {}
    }
}
```

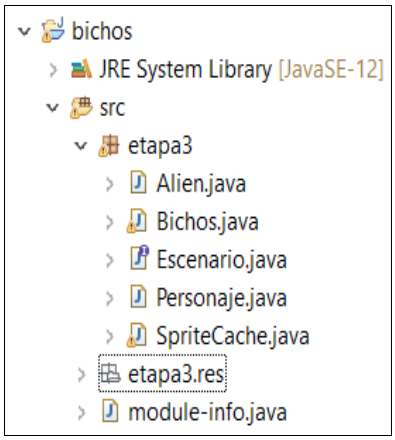
```
public Bichos() {
    sc = new  SpriteCache();
    JFrame ventana = new JFrame("Bichos");
    JPanel panel = (JPanel)ventana.getContentPane();
    setBounds(0, 0, Escenario.ANCHO, Escenario.ALTO);
    panel.setPreferredSize( new Dimension(Escenario.ANCHO, Escenario.ALTO) );
    panel.setLayout(null);
    panel.add(this);
    ventana.setBounds(0, 0, Escenario.ANCHO, Escenario.ALTO);
    ventana.setVisible(true);
    ventana.addWindowListener( new WindowAdapter() {
                                    public void windowClosing(WindowEvent e) {
                                        System.exit(0);
                                    }
                                }
                            );
    ventana.setResizable(false);
    createBufferStrategy(2);
    strategy = getBufferStrategy();
    requestFocus();
  }
```

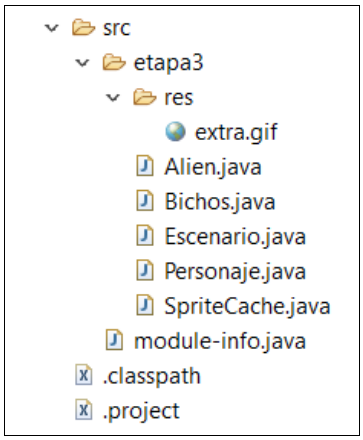
```
    public static void main(String[] args) {
        Bichos b = new Bichos();
        b.juego();
    }
}   // fin clase Bichos
```

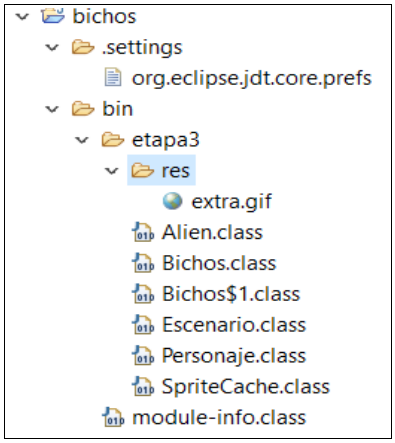
Prueba el programa a ver si funciona. En Eclipse yo he tenido problemas al cargar los recursos (imágenes, etc.). Te lo comento por si te ocurre también a tí:

```
Console ⊠  Console  Progress  Declaration  Structure
<terminated> Bichos [Java Application] D:\java\jdk12\bin\javaw.exe (7 mar. 2021 11:29:30)
Error al cargar imagen res/extra.gif de null
ERROR: java.lang.IllegalArgumentException input == null!
```

El proyecto se queda estructurado de esta manera: la vista de paquetes (nivel lógico) a la izquierda. La vista de ficheros, de los ficheros fuente (nivel físico) en el centro y vista de ficheros del programa que se genera (imagen derecha):







Pero la línea que crea la url en el método **cargaImagen(nombre)** no me funcionaba al ejecutar la aplicación en Eclipse y siempre devuelvía null. Sin embargo, al ejecutarlo en el exterior lo que no funciona es mi solución. Por eso he decidido intentarlo de las dos formas.

Muestra una captura de la ventana de la aplicación ejecutándose con el nombre "Bichos35.png".

PASO 3.6: MEJORAR LAS ANIMACIONES.

Vamos a mejorar las animaciones de los personajes. La animación de los monigotes suele ser sinónimo de fotogramas: la imagen del personaje va recorriendo una serie sucesiva de diferentes dibujos llamados "fotogramas". Cuantos más fotogramas hay, más suave es la animación del personaje. Para el alien vamos a utilizar los siguientes dos fotogramas, llamados "extra0.gif" y "extra1.gif". Debes copiarlos a la carpeta del package "etapa3/res":

Como es algo que queremos usar en todos los personajes, añadiremos esa funcionalidad a la clase Personaje. La propiedad spriteNombre desaparece y es sustituida por la propiedad **spriteNombres**, que será un array de nombres de gráficos en lugar de un único gráfico. Adicionalmente, el método **actua()** se encarga de incrementar el número de fotograma que hay que dibujar cada vez:

```
protected String[] spriteNombres;  // Sustituye a spriteNombre
protected int actualFrame;         // Fotograma a dibujar
```

```
public Personaje(Escenario e) {
    this.escena = e;
    sc = e.getSpriteCache();
    actualFrame = 0;
}
```

```
public void setSpriteNombres(String[] s) {
    spriteNombres = s;
    alto = 0;
    ancho = 0;
    for(int i = 0; i < s.length; i++ ) {
        BufferedImage imagen = sc.getSprite( spriteNombres[i] );
        alto = Math.max( alto, imagen.getHeight() );
        ancho = Math.max( ancho, imagen.getWidth() );
    }
}
```

```
public void paint(Graphics2D g){
    g.drawImage( sc.getSprite( spriteNombres[actualFrame]), x, y, escena);
}

public void actua() {
    actualFrame = (actualFrame + 1) % spriteNombres.length;
}
```

Como ahora tenemos funcionalidad en el método **actua()** de Personaje, los hijos no pueden sobreescribir ese método sin más (perderían esa funcionalidad), sino que deben llamar al método del padre antes de comenzar a realizar sus propias acciones. Esto hace que el método **actua()** la clase Alien quede modificada de la siguiente forma:

```
public void actua() {
    super.actua();
    x += vx;
    if( x < 0 || x > Escenario.ANCHO - 10 ) vx = -vx;
}
```

PASO 3.7: CONTINUAMOS CON LA MEJORA DE LAS ANIMACIONES.

Si nuestro ordenador es suficientemente rápido, los fotogramas avanzan a mucha velocidad y apenas se ven. Sin embargo, la frecuencia con la que se llama al método **actua()** es algo que es general para todo el juego y no podemos variarlo para cada personaje. La solución la tomamos prestada de la electrónica mediante lo que se llama **una base de tiempos**. El bucle principal duerme durante un tiempo indicado por el máximo común divisor de los tiempos del resto de objetos.

Por ejemplo imagina que estamos haciendo un videojuego en el que hay monstruos que actúan a la velocidad de 1 vez por segundo, mientras que el jugador actúa 2 veces por segundo (dicho de otra forma, queremos que tenga un "turno" para hacer cosas 2 veces por segundo, ó 1 vez cada medio segundo). Tenemos también un volcán que hace erupción cada 10 segundos. El máximo común divisor de 1, 0.5 y 10 es 0.5. Este MCD recibe el nombre de unidad de tiempo o base de tiempos y es el tiempo que dormirá en general el bucle principal.

Dentro del bucle principal, cada vez que salgamos del **Thread.sleep()**, se incrementa un contador secuencial que hace las veces de "instante del tiempo", como un reloj. Cuando este contador sea divisible por la cantidad apropiada, entonces es cuando se realiza la actualización del objeto en cuestión. Esta "cantidad apropiada" es el resultado de dividir el intervalo en segundos entre dos movimientos sucesivos del objeto por el MCD calculado anteriormente. Como en nuestro ejemplo la unidad de tiempo es de 0.5 segundos, el esquema sería:

- Jugador - Se actualiza siempre (0.5 / 0.5 = 1)
- Monstruo - Se mueve cada 2 unidades de tiempo, por lo tanto se actualiza cada vez que el contador sea divisible por 2.
- Volcán - Entra en erupción cada 10 segundos, o cada 20 unidades de tiempo, por lo que se actualiza cada vez que el contador sea divisible por 20.

El mismo criterio podemos aplicar a los fotogramas: mantenemos dentro de la clase Personaje un contador de tiempos y se actualiza el fotograma sólo cuando este contador sea divisible

por una cantidad que llamaremos "frameVelocidad". Con este razonamiento, la clase "Personaje" quedaría como sigue:

```
protected int frameVelocidad;   // Añadimos las variables de instancia
protected int t;                // El contador/reloj de frames
```

```
// Creamos un setter de frrameVelocidad
public void setFframeVelocidad(int fv) { frameVelocidad= fv; }
```

```
public void actua() {
      t++;
      if( t % frameVelocidad == 0 ){
          t = 0;
          actualFrame = (actualFrame + 1) % spriteNombres.length;
      }
}
```

Los descendientes de esta clase tendrían que preocuparse de establecer su propia velocidad de fotogramas. Por ejemplo la clase Alien quedaría así:

```
public Alien(Escenario e) {
  super(e);
  setSpriteNombres( new String[] { "extra0.gif", "extra1.gif"} );
  setFrameVelocidad(35);
}
```

PASO 3.8: EL JUGADOR.

No es más que un personaje que además puede ser controlado por el usuario mediante el teclado o el ratón. Vamos a hacerlo descender de Personaje y además vamos a adoptar las siguientes premisas en cuanto a su comportamiento, para poder modelar la clase:

- La nave del jugador se puede mover en todas las direcciones.
- La velocidad de movimiento del jugador es siempre la misma, aunque cambia la dirección
- Para que la nave se mueva en una dirección, la tecla correspondiente debe estar pulsada

De estas tres premisas queda claro que la nave del jugador es similar a un alien, con la única excepción de que tiene una velocidad x e y. Para el gráfico vamos a usar un único fotograma. (falta incorporar el control del movimiento):

```
package etapa3;

public class Jugador extends Personaje {
      protected int vx;
      protected int vy;
```

```
public Jugador(Escenario e) {
  super(e);
  setSpriteNames( new String[] {"nave.gif"});
}
```

```
public void actua() {
    super.actua();
    x+= vx;
    y+= vy;
    if (x < 0 || x > Escenario.ANCHO - 10) vx = -vx;
    if (y < 0 || y > Escenario.ALTO) vy = -vy;
}
```

```
      public int getVx() { return vx; }
      public void setVx(int i) {vx = i; }
      public int getVy() { return vy; }
      public void setVy(int i) {vy = i; }
 }  // fin de clase Jugador
```

Hay que modificar la clase principal para incorporar al jugador. No añadiremos el jugador a la lista de personajes del escenario porque va a tener un tratamiento especial. Sin embargo, las

cosas que se hacen con él serán las mismas - pintarlo, decirle que actúe, etc.

```
private Jugador jugador;     // Añadimos una variable miembro
```

```
public void iniciaMundo() {
    :
    // Añadimos al final del método, fuera del bucle
    jugador = new Jugador(this);
    jugador.setX( Escenario.ANCHO / 2);
    jugador.setY( Escenario.ALTO - 2 * jugador.getAlto() );
    jugador.setVx(5);
}
```

```
public void actualizaMundo() {
    :
    // Añadimos al final del método, fuera del bucle
    jugador.actua();
}
```

```
public void pintaMundo() {
    :
    // Añadimos al final del método, fuera del bucle
    jugador.paint(g);
    g.setColor(Color.green);
    :
}
```

PASO 3.9: AÑADIR CONTROL DEL USUARIO AL JUGADOR.

Añadimos ahora la posibilidad de controlar la nave del jugador con el teclado, utilizando las teclas del cursor. Aunque podríamos hacerlo en la clase Bichos, lo dejaremos la gestión del evento a la clase **Jugador** con este planteamiento:

1. La clase Bichos recibe todos los eventos de teclado.
2. Cuando ocurra un evento, la clase primero procesará las teclas que tengan que ver con el juego en general (en el caso de haberlas), como por ejemplo "pausar", "salir" o similares (de momento no tenemos ninguna).
3. Cuando hayamos determinado que la tecla pulsada no interesa al juego en general, le pasaremos los datos de esta tecla al jugador para que actúe como crea oportuno.

Definiremos dos nuevos método en la clase Jugador llamados keyPressed(...) y keyReleased(...), que serán los encargados de recibir las notificaciones de que ha sido pulsada/soltada una tecla.

En la clase Bichos:

```
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
```

```
public class Bichos extends Canvas implements Escenario, KeyListener {
    :
        // Al final del constructor
        addKeyListener(this);
    }
```

```
// Añadimos los métodos de keyListener
public void keyPressed(KeyEvent e) {  jugador.keyPressed(e);  }
public void keyReleased(KeyEvent e) { jugador.keyReleased(e); }
public void keyTyped(KeyEvent e) {}
```

Y en la clase Jugador:

```
import java.awt.event.KeyEvent;
```

Añadimos las variables miembro:

```
protected static final int VELOCIDAD_JUGADOR = 4;
private boolean sub, baj, izq, der;   // subir, bajar, izquierda y derecha
```

Y añadimos los métodos:

```
protected void actualizaVelocidad() {
    vx= 0; vy= 0;
    if(baj) vy = VELOCIDAD_JUGADOR;
    if(sub) vy = -VELOCIDAD_JUGADOR;
    if(izq) vx = -VELOCIDAD_JUGADOR;
    if(der) vx = VELOCIDAD_JUGADOR;
}
```

```
public void keyReleased(KeyEvent e) {
    switch( e.getKeyCode() ) {
    case KeyEvent.VK_DOWN:  baj = false; break;
    case KeyEvent.VK_UP:    sub = false; break;
    case KeyEvent.VK_LEFT:  izq = false; break;
    case KeyEvent.VK_RIGHT: der = false; break;
    }
    actualizaVelocidad();
}
```

```
public void keyPressed(KeyEvent e) {
    switch( e.getKeyCode() ) {
    case KeyEvent.VK_UP:    sub = true; break;
    case KeyEvent.VK_LEFT:  izq = true; break;
    case KeyEvent.VK_RIGHT: der = true; break;
    case KeyEvent.VK_DOWN:  baj = true; break;
    }
    actualizaVelocidad();
}
```

PASO 3.10: ALGO DE VIOLENCIA NUNCA VIENE MAL (EN UN JUEGO).

Añadimos la posibilidad de que el jugador dispare. De momento dejaremos el efecto que los disparos producen sobre el entorno (en particular los aliens) para más adelante. Los disparos tendrán el siguiente comportamiento:

1. Aparecen justo encima de la nave del jugador.
2. Se mueven hacia arriba, siempre a la misma velocidad.
3. El jugador sólo puede disparar cinco misiles cada vez.

Desde el punto de vista del programa, los disparos son personajes. Lo novedoso es que los disparos aparecen y desaparecen sobre la marcha y dependiendo de circunstancias que el bucle principal del programa no controla. Por ejemplo, cuando se pulsa la barra espaciadora, aparece un disparo. Este personaje debe añadirse a la lista, pero la gestión de las teclas la hace la clase Jugador, y no sería correcto que esta clase "metiera la mano" dentro de la lista de personajes, ya que esta lista es una estructura de datos propia de la clase Bichos (un Escenario). Una solución es dotar al escenario de métodos que permitan añadir y quitar personajes sobre la marcha. Con esta idea, a la interfaz Escenario añadimos:

```
public void insertaPersonaje(Personaje p);
```

Hay un problema que no es obvio a primera vista: ¿quién borra los personajes? Lo primero que se nos podría ocurrir es añadir un método **borraPersonaje(...)** igual que el de insertar. Pero surge un problema y es que es posible que el borrado de un personaje se haga como respuesta a la pulsación de una tecla, o bien como consecuencia de un método actua() de otro personaje. En el primer caso, el borrado se estaría produciendo desde un hilo distinto al principal (nada bueno ya que estaríamos manipulando una estructura de datos desde un hilo, mientras que desde el principal tal vez la estemos recorriendo). En el segundo caso, estaríamos en una situación similar - podría producirse un desajuste en el bucle y que el método actua() de algún personaje no se llame. Para evitar ambas situaciones, en lugar de borrar un personaje

directamente, definimos un centinela que indique que un personaje determinado está "marcado para ser borrado". Posteriormente será el bucle principal el único que comprobará dicha marca y eliminará el personaje si procede. Esto requiere una modificación a la clase Personaje de la siguiente forma:

```
//Añadimos una variable de instancia a cada objeto Personaje
protected boolean marcadoParaBorrar;

// Añadimos getters y setters
public void borra() { marcadoParaBorrar = true; }
public boolean estaMarcadoParaBorrar() { return marcadoParaBorrar; }
```

En la clase Bichos, añadimos insertaPersonaje() para cumplir con la interfaz:

```
public void insertaPersonaje(Personaje p) { personajes.add(p); }
```

```
public void actualizaMundo() {  // Modificar actualizaMundo()
    int i = 0;
    while( i < personajes.size() ) {
        Personaje p = (Personaje)personajes.get(i);
        if( p.estaMarcadoParaBorrar() ) {
            personajes.remove(i);
        } else {
            p.actua();
            i++;
        }
    }
    jugador.actua();
}
```

Una vez que hemos resuelto los problemas de añadir y eliminar personajes, podemos comenzar con los disparos. Para el disparo utilizaremos el siguiente gráfico, llamado "misil.gif".

```
package etapa3;

public class Misil extends Personaje {
    protected static final int VELOCIDAD_MISIL = 10;

    public Misil(Escenario e) {
        super(e);
        setSpriteNombres( new String[] {"misil.gif"});
    }

    public void actua() {
        super.actua();
        y -= VELOCIDAD_MISIL;
        if (y < 0) borra();
    }
}
```

Por último, queda añadir la gestión de la tecla de disparo a la clase Jugador. Añade al switch del método keyPressed():

```
// Al pulsar la barra del espacio
case KeyEvent.VK_SPACE: disparar(); break;
```

```
public void disparar() {
    Misil m = new Misil(escena);
    m.setX(x);
    m.setY(y - m.getAlto());
    escena.insertaPersonaje(m);
}
```

PASO 3.11: LA CARRERA ARMAMENTÍSTICA NO CESA.

Vamos a comprobar las delicias de la OOP añadiendo un nuevo tipo de arma: la "bomba de fragmentación". El usuario tendrá un número limitado de estas bombas (supongamos 5) y cada vez que dispare, aparecerán ocho bolas de fuego que saldrán despedidas en las ocho direcciones posibles. Para este propósito utilizaremos los siguientes gráficos:

bombaSI.gif	bombaS.gif	bombaSD.gif
bombaI.gif		bombaD.gif
bombaBI.gif	bombaB.gif	bombaBD.gif

La bola de fuego tendrá su propia clase, llamada Bomba, con el siguiente código:

```java
package etapa3;

public class Bomba extends Personaje {
    public static final int SUBE_IZQ = 0;
    public static final int SUBE     = 1;
    public static final int SUBE_DER = 2;
    public static final int IZQ      = 3;
    public static final int DER      = 4;
    public static final int BAJA_IZQ = 5;
    public static final int BAJA     = 6;
    public static final int BAJA_DER = 7;

    protected static final int VELOCIDAD_BOMBA = 5;
    protected int vx;
    protected int vy;
```

```java
public Bomba(Escenario e, int direccion, int x, int y) {
    super(e);
    this.x = x;
    this.y = y;
    String sprite = "";
    switch(direccion) {
    case SUBE_IZQ: vx = -VELOCIDAD_BOMBA;
                   vy = -VELOCIDAD_BOMBA;
                   sprite= "bombaSI.gif";
                   break;
    case SUBE:     vx = 0;
                   vy = -VELOCIDAD_BOMBA;
                   sprite= "bombaS.gif";
                   break;
    case SUBE_DER: vx = VELOCIDAD_BOMBA;
                   vy = -VELOCIDAD_BOMBA;
                   sprite= "bombaSD.gif";
                   break;
    case IZQ:      vx = -VELOCIDAD_BOMBA;
                   vy = 0;
                   sprite = "bombaI.gif";
                   break;
    case DER:      vx = VELOCIDAD_BOMBA;
                   vy = 0;
                   sprite = "bombaD.gif";
                   break;
    case BAJA_IZQ: vx = -VELOCIDAD_BOMBA;
                   vy = VELOCIDAD_BOMBA;
                   sprite= "bombaBI.gif";
                   break;
    case BAJA:     vx = 0;
                   vy = VELOCIDAD_BOMBA;
                   sprite = "bombaB.gif";
                   break;
    case BAJA_DER: vx = VELOCIDAD_BOMBA;
                   vy = VELOCIDAD_BOMBA;
                   sprite = "bombaBD.gif";
                   break;
    }
    setSpriteNombres( new String[] {sprite} );
}
```

```java
        public void actua() {
            super.actua();
            y+= vy;
            x+= vx;
            if (y < 0 ||
                y > Escenario.ALTO ||
                x < 0 ||
                x > Escenario.ANCHO)
              borra();
        }
} // fin clase Bomba
```

Observa que el método actua() se limita a mover la bomba y borrarla si se sale de la pantalla en algún momento. La clase Jugador debe llevar el contador de las bombas de las que dispone

y para hacer que al pulsar una tecla especial (la tecla "B") se disparen las ocho bolas de fuego:

```
private int reservaBombas;     //Cantidad de bombas
```

En el constructor:

```
reservaBombas = 5;
```

Añadir el getter:

```
public int getReservaBombas() { return reservaBombas; }
```

En el switch del keypressed():

```
case KeyEvent.VK_B : disparaBombas(); break;
```

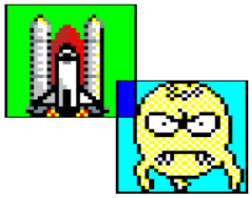
Añade el método:

```
public void disparaBombas() {
    if (reservaBombas == 0) return;  // Ya no tiene
    reservaBombas--;
    escena.insertaPersonaje( new Bomba(escena, Bomba.SUBE_IZQ, x, y) );
    escena.insertaPersonaje( new Bomba(escena, Bomba.SUBE, x, y) );
    escena.insertaPersonaje( new Bomba(escena, Bomba.SUBE_DER, x, y) );
    escena.insertaPersonaje( new Bomba(escena, Bomba.IZQ, x, y) );
    escena.insertaPersonaje( new Bomba(escena, Bomba.DER, x, y));
    escena.insertaPersonaje( new Bomba(escena, Bomba.BAJA_IZQ, x, y) );
    escena.insertaPersonaje( new Bomba(escena, Bomba.BAJA, x, y) );
    escena.insertaPersonaje( new Bomba(escena, Bomba.BAJA_DER, x, y) );
}
```

Muestra captura de ventana del juego con el nombre "Bichos310.png" donde se vean disparos y bombas y la nave del jugador por el centro de la pantalla.

PASO 3.11: DETECTAR COLISIONES DE PERSONAJES.

Hasta ahora los monstruos se rían en nuestra cara de los disparos porque ninguno de ellos les afectaba lo más mínimo. Ha llegado la hora de añadir la detección de colisiones a nuestro programa. En general es erróneo plantearse la detección de colisiones como un asunto de objetos puntuales, ya que nuestros sprites tienen tanto anchura como altura. De esta forma, habrá una colisión cuando los rectángulos que encierran cada sprite se intersecten (zona azul oscura de la siguiente figura):



Esta comprobación habrá que hacerla para *todas las posibles parejas de personajes*, y además habrá que hacerla para cada personaje con el jugador. Quien tiene la lista de personajes es Escenario, así que le toca a la clase Bichos.

Afortunadamente, la clase Rectangle de Java incorpora el método intersects() que permite comprobar si un rectángulo intersecta con otro. Como necesitaremos saber los rectángulos que engloban cada sprite, podemos añadir un método (por ejemplo getFronteras()) en la clase

Personaje que devuelva el objeto Rectangle de cada uno.

Cuando se detecta una colisión, ¿qué hay que hacer? De nuevo echamos mano del polimorfismo y de la *delegación de responsabilidades* y lo que haremos es notificar a cada personaje que se ha producido una colisión con otro y que ellos decidan lo que hacer. Esto significa que aparece un nuevo método en la clase Personaje llamado **colision(...)**. Por defecto este método no hace nada, siendo las clases hijas las responsables de sobreescribirlo como crean oportuno (podría ser un método abstracto). En la clase Personaje:

```
import java.awt.Rectangle;

public Rectangle getFronteras() { return new Rectangle(x, y, ancho, alto); }
public void colision(Personaje a){ }
```

En la clase Alien, queremos que un alien muera cuando ocurre una colisión con un misil o una bomba.

```
public void colision(Personaje a) {
    if(a instanceof Misil || a instanceof Bomba) borra();
}
```

Y finalmente queda la propia detección de colisiones, que se realiza en la clase principal, en un nuevo método al que hemos llamado **compruebaColisiones()**:

```
import java.awt.Rectangle;

public void compruebaColisiones() {
    Rectangle jugadorFronteras = jugador.getFronteras();
    for(int i = 0; i < personajes.size(); i++) {
        Personaje p1 = (Personaje)personajes.get(i);
        Rectangle r1 = p1.getFronteras();
        if( r1.intersects(jugadorFronteras) ) {
            jugador.colision(p1);
            p1.colision(jugador);
        }
        for(int j = i + 1; j < personajes.size(); j++) {
            Personaje p2 = (Personaje)personajes.get(j);
            Rectangle r2 = p2.getFronteras();
            if( r1.intersects(r2) ) {
                p1.colision(p2);
                p2.colision(p1);
            }
        }
    }
}
```

En el método juego(), tras actualizar el mundo, se comprueban las colisiones:

```
:
compruebaColisiones();
:
```

Lo siguiente que haremos es cambiar el tamaño de la superficie del juego por la que se pueda mover el jugador, ya que reservaremos una zona para mostrar el estado actual. Así que añadimos una nueva constante llamada **LIMITE_JUGADOR** a la interface Escenario. Esta constante tendrá el límite vertical del área de juego:

```
public static final int LIMITE = 400;
```

Cuando matamos un alien no ocurre nada especial y se exterminan con bastante rapidez. Podemos variar la situación de distintas formas:

- Hacer aparecer nuevos aliens cada vez que muere uno.
- Cambiar el "nivel" de juego cambiando decorados y creando bichos diferentes.
- Hacer que aparezcan nuevos alien de forma periódica al margen de los que matamos.

Utilizaremos el primero de los enfoques: cada vez que matemos un alien, aparece otro. Esto es fácil de conseguir añadiendo un método y llamándolo desde colision() de la clase Alien cuando borremos uno:

```
if( a instanceof Misil || a instanceof Bomba ) {
    borra();
    genera();
}
:
```

```
public void genera() {
    Alien a = new Alien(escena);
    a.setX( (int)(Math.random() * Escenario.ANCHO) );
    a.setY( (int)(Math.random() * Escenario.LIMITE/2) );
    int v = (int)(Math.random() * 20 - 10);
    a.setVx( (v == 0)? 1:v );
    escena.insertaPersonaje(a);
}
```

Ahora que somos capaces de matar aliens, sería bueno disponer de alguna clase de puntuación. Así que añadiremos una barra de estado en la parte inferior de la pantalla. Esta barra contendrá lo siguiente:

- El contador de fps.
- La puntuación.
- Un medidor de la vida/salud que le queda al jugador.
- Una representación gráfica del número de bombas de fragmentación que quedan.

Comenzamos primero por la puntuación y la vida. Estos atributos pertenecen a la clase del jugador, así que los añadiremos allí, junto con sus correspondientes métodos de acceso. Por otra parte, como es bastante molesto que la nave del jugador "rebote" cuando llega a los límites de la pantalla, modificamos este comportamiento y hacemos que simplemente se quede en el borde. Y en el constructor añadir:

```
public static final int MAX_VIDA = 200;
private int puntos;
private int vida;
```

```
vida = MAX_VIDA;
puntos = 0;
```

Modifica actua() y añade getters y setters de puntos y vida:

```
public void actua() {
    super.actua();
    x+= vx;
    y+= vy;
    if( x < 0 )  x = 0;
    if( x > Escenario.ANCHO - getAncho() )
        x = Escenario.ANCHO - getAncho();
    if( y < 0 ) y = 0;
    if( y > Escenario.LIMITE - getAlto() )
        y= Escenario.LIMITE - getAlto();
  }
}
```

```
public int getPuntos() { return puntos; }
public void setPuntos(int i) { puntos = i; }
public int getVida() { return vida; }
public void setVida(int i) { vida = i; }
```

El mayor cambio ocurrirá en la clase Bichos, donde ahora tendremos 4 nuevos métodos para dibujar las diferentes partes de la barra de estado:

```
import java.awt.Font;
import java.awt.image.BufferedImage;
```

Añadimos estos métodos a la clase Bichos:

```
public void pintaVida(Graphics2D g) {
    g.setPaint(Color.red);
    g.fillRect(280, Escenario.LIMITE, Jugador.MAX_VIDA, 30);
    g.setPaint(Color.blue);
    g.fillRect(280 + Jugador.MAX_VIDA - jugador.getVida(),
               Escenario.LIMITE,
               jugador.getVida(), 30);
    g.setFont( new Font("Arial", Font.BOLD, 20) );
    g.setPaint(Color.green);
    g.drawString("Vida", 170, Escenario.LIMITE + 20 );
}
```

```
public void pintaFps(Graphics2D g) {
    g.setFont( new Font("Arial",Font.BOLD,12) );
    g.setColor(Color.white);
    if( fps > 0 )
        g.drawString(String.valueOf(fps) + " fps",
                     Escenario.ANCHO - 50,
                     Escenario.LIMITE);
    else
       g.drawString("--- fps",
                    Escenario.ANCHO - 50,
                    Escenario.LIMITE);
}
```

```
public void pintaPuntos(Graphics2D g) {
    g.setFont( new Font("Arial", Font.BOLD, 20) );
    g.setPaint(Color.green);
    g.drawString("Puntos:", 20, Escenario.LIMITE + 20);
    g.setPaint(Color.red);
    g.drawString(jugador.getPuntos() + "" , 100,
                 Escenario.LIMITE + 20);
}
```

```
public void pintaEstado(Graphics2D g) {
    pintaPuntos(g);
    pintaVida(g);
    pintaArmas(g);
    pintaFps(g);
}
```

```
public void pintaArmas(Graphics2D g) {
    int xBase = 280 + Jugador.MAX_VIDA + 10;
    for(int i = 0; i < jugador.getReservaBombas(); i++) {
       BufferedImage bomba = sc.getSprite("bombaSI.gif");
       g.drawImage( bomba, xBase + i * bomba.getWidth(),
                    Escenario.LIMITE, this);
    }
}
```

En el método pintaMundo(), tras pintar el jugador, cambiamos el if que pintaba los fps por la llamada:

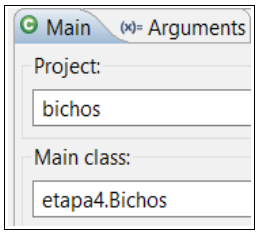
```
pintaEstado(g);
```

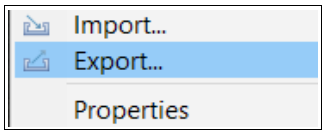
Muestra una captura de ventana de la aplicación llamada "**Bichos311.png**".

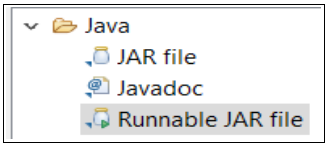
(Si quieres acabarlo, tienes una tarea opcional para darle los últimos retoques...)

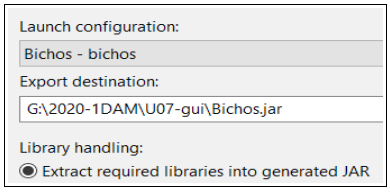
PASO 3.12: CREA UN EJECUTABLE.

Desde Eclipse, genera un .jar ejecutable (se eliminan los fuentes) y también me lo entregas. Cambia la configuración del proyecto (Project → Properties → Run/Debug Settings, seleccionas la configuración, botón Edit) e indicas la clase que se debe ejecutar (la Main class). Luego, en el menú Fichero → Export escoge dentro de Java - > Runnable JAR file e indicas la configuración que quieres usar y la ruta de destino.









Me pasas el .jar generado.

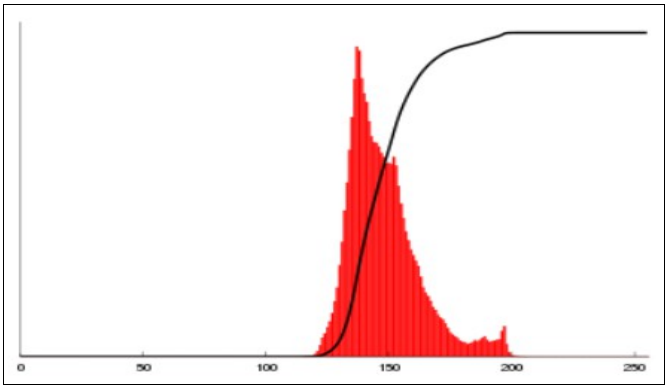
EJERCICIO 5 (1 PUNTO). ECUALIZAR IMÁGENES.

Una imagen está formada por una tabla de números enteros en la que cada elemento contiene el nivel de color (RGB) de un píxel. En las imágenes sin color, los 3 componentes RGB tienen el mismo valor (R = G = B) y este valor puede ir de 0 (negro) a 255 (blanco).

En realidad, si trabajamos con una BufferedImage tendremos todos los píxels en un array de una única dimensión, aunque podemos usar los métodos **int getRGB(int x, int y)** y **void setRGB(x, y, int RGB)** para leer y cambiar respectivamente los valores de los píxels como si estuviesen en dos dimensiones.

Algunas imágenes tienen mala calidad porque los diferentes colores se concentran en una franja muy estrecha. De forma que o son muy oscuras, o muy claras (poco contraste). Esto se puede deber a fuentes de luz, condiciones atmosféricas (niebla, lluvia, etc.) o ambientales (sitios oscuros, la hora del día cuando se toma la fotografía, etc.)

Ejemplo: Esta fotografía de un paisaje se ha tomado en condiciones de niebla. A la derecha se ve su histograma (en rojo) y su histograma acumulado (la línea negra). Si lo observas con atención, verás que la zona roja se acumula en el rango de niveles de gris entre 140 y 200. Esto significa que tiende a ser una imagen clara y con pocos niveles de gris diferentes.



Ecualizar el histograma de una imagen consiste en intentar que tenga más diversidad de niveles y es un procesamiento que puede mejorar la calidad en ciertas condiciones. El proceso te lo explico a continuación con un ejemplo sencillo. Imagina que tenemos una imagen de 3x4 píxels con los niveles de gris (RGB) que aparecen. Un 3 significa color (3,3,3).

3	4	5	6
5	4	3	6
5	6	7	5

3	4	5	6	5	4	3	6	5	6	7	5

En un buffer se guardaría en una sola dimensión y cada color codificado en un int (R * 256 * 256 + G * 256 + B)

Calcular el histograma consiste en recorrer todos los píxels y contar qué cantidad de píxels hay de cada nivel de gris. Teniendo en cuenta que hay 256 posibles valores de 0 (negro) a 255 (blanco) se necesita un array de 256 doubles.

```
double histo = new double[256];
```

```
para cada pixel p de la imagen
    nivel = nivel de gris del pixel p;  // Un nº entero entre 0 y 255
    histo[nivel] = histo[nivel] + 1;
```

Además necesitas saber el histograma acumulado que es cuantos píxels tienen un nivel de gris inferior o igual al actual. Es decir:

```
histoAcu[0] = histo[0]
histoAcu[1] = histoAcu[0] + histo[1]
histoAcu[2] = histoAcu[1] + histo[2]
histoAcu[3] = histoAcu[2] + histo[3]
   :              :              :
```

En el ejemplo:

histo	0	0	0	2	2	4	3	1	0	0	0	0	...	0
nivel	0	1	2	3	4	5	6	7	8	9	10	11	...	255

histoAcu	0	0	0	2	4	8	11	12	12	12	12	12	...	12
nivel	0	1	2	3	4	5	6	7	8	9	10	11	...	255

Ahora, para sacar la frecuencia relativa de cada nivel de gris en la imagen, dividimos tanto los valores de histo como los de histoAcu por el valor histoAcu[255] (coincide con la cantidad de píxels que hay en la imagen). Y nos darían estos valores:

histo	0,00	0,00	0,00	0,17	0,17	0,33	0,25	0,08	0,00	0,00	0,00	0,00	...	0
nivel	0	1	2	3	4	5	6	7	8	9	10	11	...	255

histoAcu	0,00	0,00	0,00	0,17	0,33	0,67	0,92	1,00	1,00	1,00	1,00	1,00	...	1,00
nivel	0	1	2	3	4	5	6	7	8	9	10	11	...	255

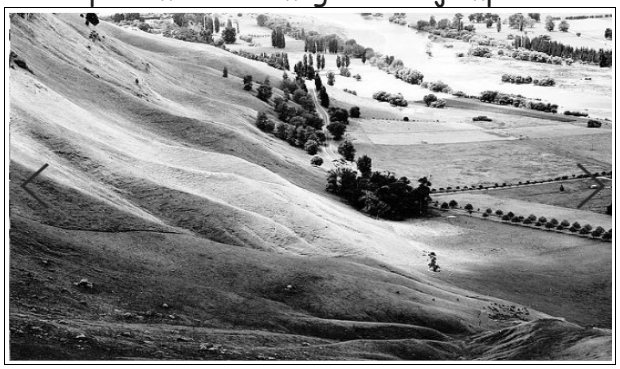
El método **BufferedImage ecualizar(BufferedImage img)** debe conseguir crear y devolver una versión ecualizada de la imagen img donde su histograma sea más uniforme, cambiando la luminosidad de los píxels para expandir más niveles de gris, para que no estén tan concentrados en un solo intervalo. Así, conseguimos aumentar el contraste de la imagen. Para ecualizar una imagen, se transforma cada píxel de tal forma que, si tenemos un píxel con el nivel de gris i, éste pasará a tener el nivel de gris:

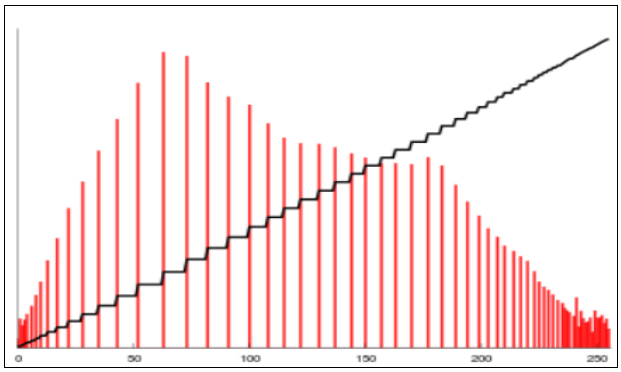
255 * histoAcu[i] / histoAcu[255]

Si lo aplicamos a la pequeña imagen de ejemplo:

4	7	14	19
14	7	21	19
14	19	21	14

Si lo aplicamos a la imagen del ejemplo:



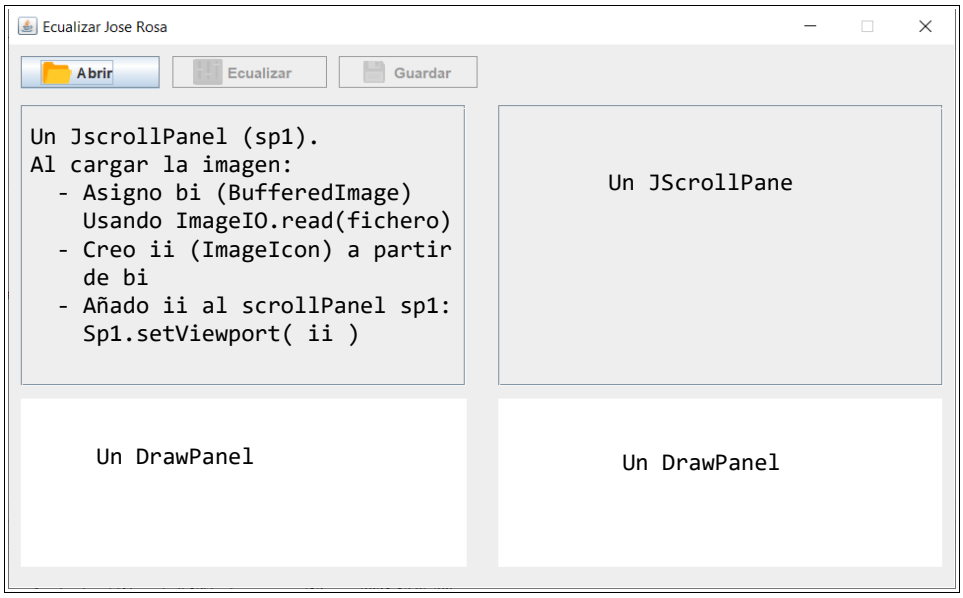


Haz el programa E705 que (tienes libertad para usar la interfaz que quieras) y pasas el ejecutable, donde:

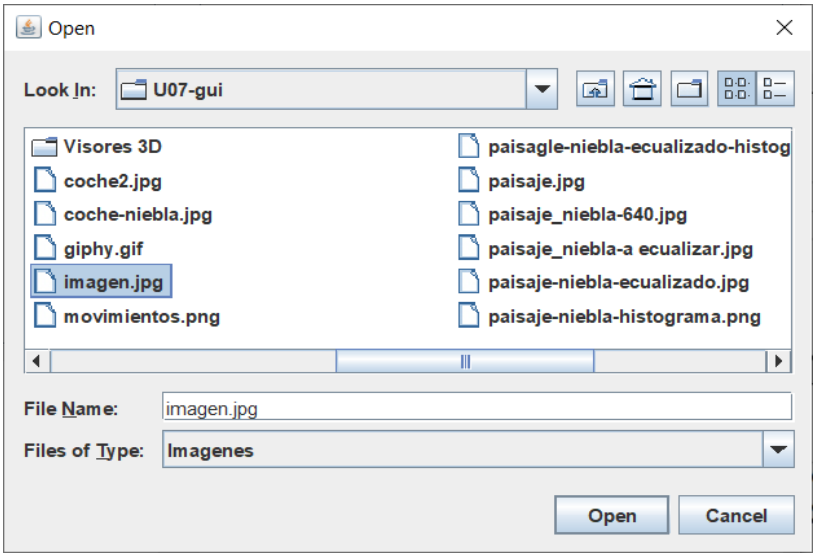
- Aparezca tu nombre y primer apellido en la barra de título de la ventana.
- Use un JFileChooser para seleccionar una imagen.
- Si es en color la transforma en niveles de grises.
- La muestra con un tamaño fijo (la dibujas escalada).
- Calcula su histograma e histograma acumulado y lo dibujas en un gráfico similar al de las imágenes de ejemplo (incluidos los ejes.
- Usa un botón que permita ecualizarla, mostrando la imagen generada y de nuevo el gráfico de su histograma y acumulado.

Ejemplo: Yo he usado WindowBuilder para diseñar la interface. Esto tiene el problema de que es complicado pintar en los JPanel y que el dibujo no se pierda al mover/minimizar/maximizar/solapar/redimensionar la ventana. Mi solución ha sido sustituir la creación de los JPanel, por una versión de una clase hija definida por mí, a la que sobreescribo el método paint y ahí dibujo el histograma (aunque eso tiene la consecuencia de que Window Builder ya no te deja cambiarle muchas de sus propiedades en tiempo de diseño):

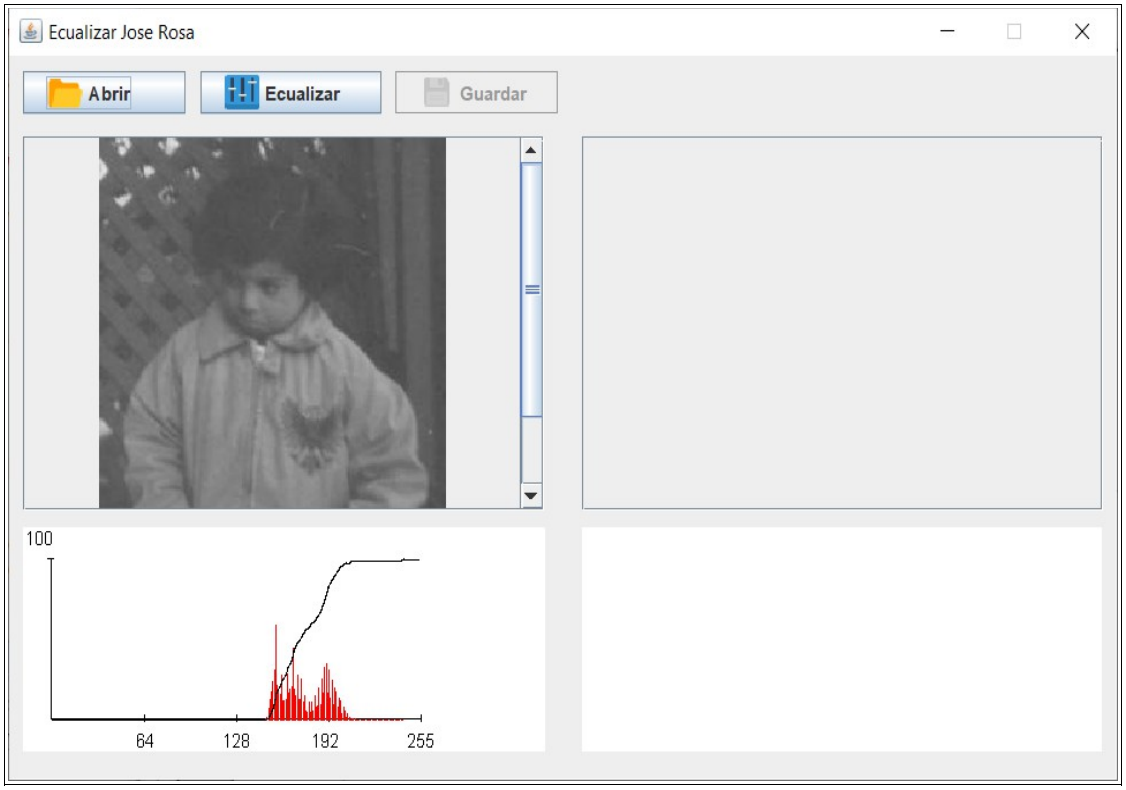
```
public class DrawPanel extends Jpanel {
     // Las dejo públicas porque la clase solo se utiliza en el Frame
     public BufferedImage b;   // La imagen a la que saco el histograma.
     public h:                 // El array del hisograma
     public ha;                // El array del histograma acumulado

    @Override
    public paint(Graphics g) {
        super.paint(g);
        g.setColor(Color.white);
        g.drawFillRect(0,0, getWidth(), getHeight() );
        if( b == null || h == null; || ha == null ) return;
        // Aquí dibujo el histograma h e histograma acumulado ha ya caculados fuera...
    }
```



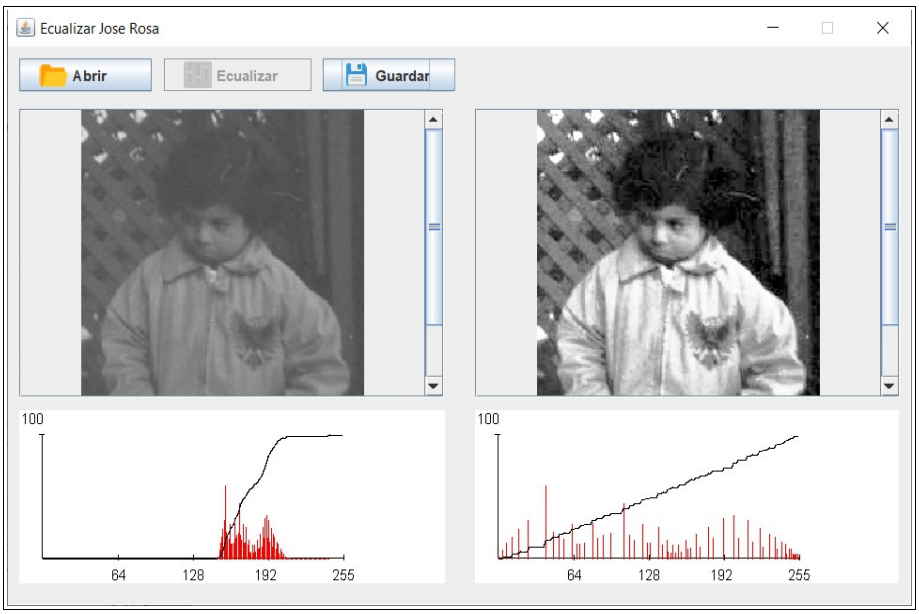
Si pulso el botón Abrir:



Si escojo una imagen:



Si pulso el botón ecualizar:



EJERCICIOS JEDI (EL QUE TE TOQUE)

Como siempre, aunque solo debes entregar el que te toca, deberías intentarlos todos para practicar!!

EJERCICIO 6 (1 PUNTO). DIBUJAR UN GRÁFICO DE DATOS.

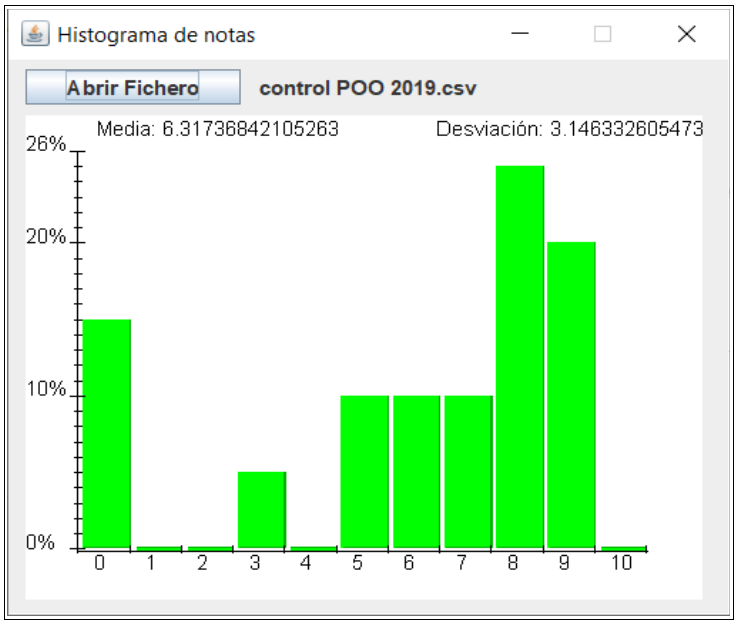
Haz el programa ejecutable **U7E6.jar** que calcule y dibuje el histograma de las notas almacenadas en un fichero .csv que tiene este formato:

```
Nombre de la prueba   // Una línea opcional, podría no estar
alumno; nota          // Un nº indeterminado de líneas con este formato
```

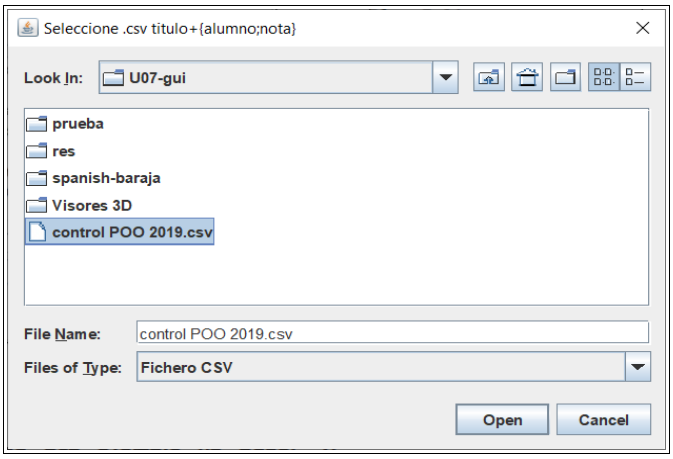
El programa debe:

- Poner tu nombre y primer apellido en el título de la ventana.
- Permitir localizar ficheros .csv usando un filtro y un JFileChooser.
- Comprobar el formato de las líneas (la primera el título y las siguientes alumno y nota) detectando/ignorando las líneas que no tengan estos dos campos.
- Las conversiones de texto a double de las notas hazlas de forma protegida y si da error debido al uso del punto o coma decimal, debes solucionarlo.
- Además debe calcular y mostrar la nota media y la desviación estándar.

A modo de ejemplo:




```
Control de POO Avanzada
Epi;9,49
Blas;7,04
Coco;8,20
Pepa Pig;9,08
Abeja Maya;6,33
Doraimon;8,61
Nobita;3,57
Lisa;7,39
Bart;5,15
Hommer;0,00
Pocoyo;0,00
Pajaroto;9,24
Pajarito;6,14
Eli;0,00
Pato;8,05
Daisy;8,66
Lucas;9,29
Silvestre;5,36
Piolín;8,43
```



Nota: *para poder dibujar usa por ejemplo un panel y sobreescribe su método paint() modificando el código que genere WindowBuilder.*

```
panel = new JPanel() {
   public void paint(...) {...}
};
```

EJERCICIO 7. DIBUJAR FRACTALES.

Crea la aplicación ejecutable U7E7.jar que muestre un grupo de dos radios agrupados con dos posibilidades con etiquetas "Ecuación recurrente" e "IFS", y un botón "Dibujar" y debajo un panel donde puedas dibujar uno u otro gráfico.

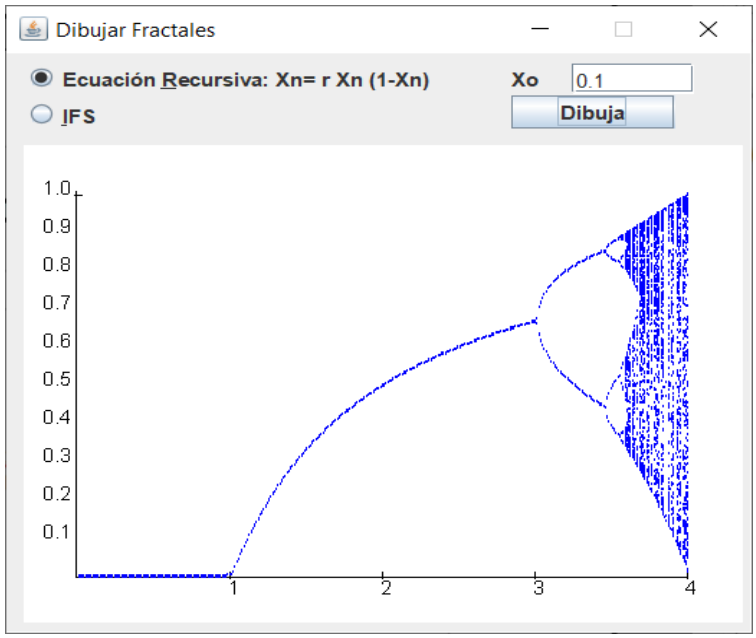
Ecuación recurrente: Desde el siglo XIX se usa esta ecuación para modelar por ejemplo, el crecimiento de una población (en realidad para muchas cosas más, mira el siguiente vídeo https://www.youtube.com/watch?v=EOvLhZPevm0).

$$x_{n+1} = r x_n (1 - x_n)$$

Donde x_n es un valor entre 0 y 1 y representa el porcentaje de individuos que tiene una población con respecto al máximo que un ecosistema puede mantener. X_0 es el porcentaje inicial de esa población.

El factor r representa la fertilidad o tasa de crecimiento de la población. De forma que pasado cierto período de tiempo (se usa un año normalmente), la población aumentará en ese factor. Como la cantidad de población está limitada, el segundo factor ($1 - X_n$) se encarga de corregir el valor para que no supere el límite. Por ejemplo: si tenemos un ecosistema con una tasa de conejos de 0.1 y un ratio de crecimiento de 2.9, ¿Cuántos conejos habrá cuando pasen muchos años?

Si has visto el vídeo que te propongo, verás que la evolución de la población a largo plazo depende más de la tasa de crecimiento (r) que de la población inicial X_0. De hecho ocurre que con tasas r entre 0 y 3, la población acaba estabilizándose en un único valor. Se dice que alcanza un **punto fijo** o **atractor** en que **x = f(x)**.

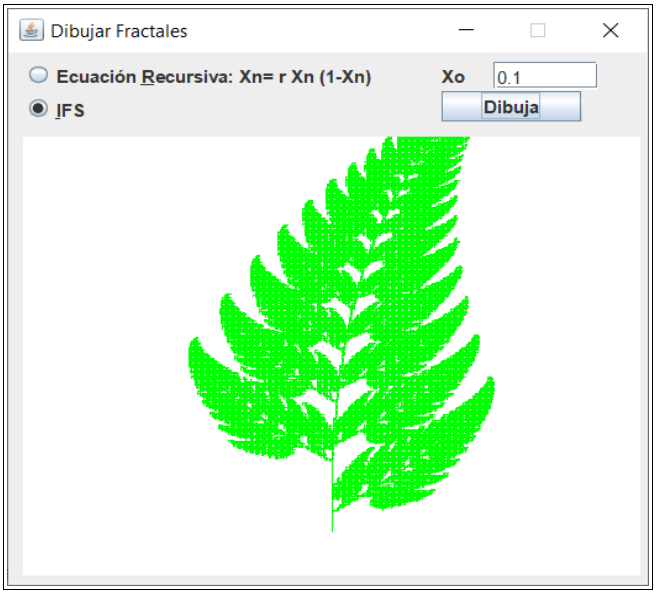


Pero para ciertos valores de r mayores de 3, las poblaciones van cambiando en dos valores (un año v1, otro v2, v1, v2, …). Es como una bifurcación. Cuando se alcanza otro valor entre 3 y 4, evolucionan en 4 valores (v1, v2, v3, v4), … luego 8,…

Y para algunos valores de r, aparece el caos, nunca evolucionan de la misma forma, nunca se estabilizan y son impredecibles (de hecho se usó para generar nºs aleatorios porque pasaba los test estadísticos). Este comportamiento se describe en **el diagrama de bifurcaciones** que partiendo de un valor inicial (X_0) representa como evoluciona la población para diferentes r entre 0 y 4. Si se elige esta opción, se hace visible una caja de texto donde se permite escribir un X_0 (por defecto tendrá el valor 0.1) y dibujar el diagrama de bifurcaciones al pulsar el botón. El código debe comprobar que el valor está entre 0 y 1, si no es así pon el fondo de la caja X_0 en rojo. Si cumple la condición, pon el fondo a blanco y dibuja el diagrama en el panel de 400 x 400 píxels (incluye ejes y etiquetas de los ejes para identificar escalas y deja un margen de 30 píxels en los 4 lados). Si tienes en cuenta que en el eje Y los valores irán de 0 a 1, cada píxel representa un valor que está a una distancia de 0.01 del siguiente. Algoritmo en pseudocódigo para generar el diagrama:

```
// Si cada distancia de 1 tiene 100 píxels, [0-4] son 400 píxels.
Borra gráfico anterior, dibuja ejes y pone sus etiquetas
PARA i DESDE 0 HASTA 400 HACER
    r = i / 100;
    SI X0 NO CUMPLE 0 < x < 1
    ENTONCES
       poner fondo rojo
       salir
    FIN SI
```

```
        x = … valor leído de la caja de edición que cumple 0 < x < 1;
        PARA j DESDE 0 HASTA 1000 HACER x = r * x * (1 – x); FIN PARA
        PARA j DESDE 0 HASTA 100 HACER
           x= r * x * (1 - x);
           Dibuja el punto en el gráfico (i, x);
        FIN PARA
    FIN PARA
```



Nota: escalado a 1/8. Es decir si se generan los valores (x,y), se dibujan (x/8, y/8) suponiendo que (0,0) está en el centro del panel.

IFS DE BARNLEY FERM. Los sistemas de funciones iteradas (IFS) como describe el teorema del collage, son un grupo de funciones que al aplicarlas de forma iterada son capaces de regenerar estructuras complejas (imágenes en este caso) a partir de pocos datos. Se usa en técnicas de compresión de imágenes con pérdidas, usando fractales. Este caso concreto usa 4 transformaciones afines definidas así:

$$f(x,y) = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix}$$

Las funciones quedan resumidas en esta tabla:

w	a	b	c	d	e	f	p	Portion generated
f_1	0	0	0	0.16	0	0	0.01	Stem
f_2	0.85	0.04	−0.04	0.85	0	1.60	0.85	Successively smaller leaflets
f_3	0.20	−0.26	0.23	0.22	0	1.60	0.07	Largest left-hand leaflet
f_4	−0.15	0.28	0.26	0.24	0	0.44	0.07	Largest right-hand leaflet

El proceso de reconstruir la imagen comienza en el punto X_0= (0,0) y aleatoriamente se elige una de las siguientes transformaciones:

f1: Xn+1 = 0
Yn+1= 0.16 Yn
Se escoge el 1% de las veces y define el tallo.

f2: Xn+1 = 0.85 Xn + 0.04 Yn
Yn+1 = -0.04 Xn + 0.85 Yn + 1.6
Se escoge el 85% de las veces y define la parte externa de las hojas.

f3: Xn+1 = 0.2 Xn – 0.26 Yn
Yn+1 = 0.23 Xn + 0.22 Yn + 1.6
Se escoge el 7% de las veces y define la parte interna de cada hoja

f4: Xn+1 = -0.15 Xn + 0.28 Yn
Yn+1 = 0.26 Xn + 0.24 Yn + 0.44

Se escoge el 7% de las veces, define la parte interna de las hojas hasta el tallo.

Algoritmo en Pseudocódigo:

```
Borrar el área de dibujo con fondo blanco. Poner a dibujar en color verde.
x ← 0;   y ← 0;
PARA i DESDE 0 HASTA 200000 HACER
    r ← nº aleatorio entre [0, 1);
    SI r <= 0.01 ENTONCES cambiar x e y con f1
    SINO SI r <= 0.86 ENTONCES cambiar x e y con f2
         SINO SI r <= 0.93  ENTONCES cambiar x e y con f3
              SINO cambiar x e y con f4;
    Dibujar el punto (x,y) de color verde sobre fondo blanco
FIN PARA
```

EJERCICIO 8 (1 PUNTO + 1 OPCIONAL): MANDELBROT.

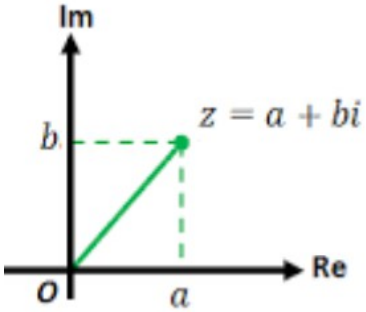
Haz la aplicación gráfica **U7E8.jar** donde aparezca tu nombre y primer apellido en la barra de título que dibuje el conjunto de Mandelbrot. Además, si haces clic en algún punto del gráfico de la pantalla con el botón izquierdo del ratón, el programa calcula a qué punto del gráfico corresponde el píxel que has clicado y vuelve a dibujar el conjunto, pero poniendo ese punto en el centro del dibujo y haciendo un zoom x2. Si haces clic con el botón derecho, vuelves al nivel de zoom anterior y vuelves a dejar como centro el anterior centro, salvo que estés en el clic original en cuyo caso no hace nada.

El conjunto de Mandelbrot es un fractal que representa como se comporta cada punto del plano de los números complejos cuando iteras muchas veces una misma función. Si llamamos c al número complejo del que queremos saber su comportamiento, le aplicamos esta transformación repetidamente:

$$Zn+1 = Zn * Zn + c$$

Donde Zn es otro número complejo. Si al hacer la transformación muchas veces, los resultados no tienden hacia el infinito, el punto pertenece al conjunto de Mandelbrot y se dibuja el punto de color negro. Pero si el punto crece hacia el infinito, el punto no pertenece al conjunto y se pinta de un color que indique cuando se ha acercado al infinito.

Recuerda que un número complejo tiene dos partes: la parte real corresponde a la coordenada en el eje X (horizontal) y la parte imaginaria que corresponde a la coordenada del eje Y (vertical).



La letra i simboliza la raíz cuadrada de -1, que no tiene solución real (por eso es imaginario). Para que sepas hacer las operaciones, si tenemos dos complejos Z1 y Z2 donde:

Z1 = a + bi	Z2 = c + di
Suma de z1 y z2:	z1 + z2 = (a + c) + (b + d)i
Multiplicación de z1 y z2:	z1 * z2 = (ac - bd) + (ad + bc) i

Aquí tienes pseudocódigo y un par de enlaces (https://youtu.be/u1pwtSBTnPU y https://youtu.be/8cgp2WNNKmQ) por si quieres ver lo curioso que es este conjunto que tiene muchas partes autosemejantes. En pseudocódigo:

```
// **** CONFIGURACIÓN INICIAL
// El panel donde dibujes el conjunto tiene un tamaño de (ancho, alto) y fondo negro
// Define un array de hasta 100 colores que irás generando a medida que los necesites
// Define el tamaño del plano de los complejos que vas a dibujar y el centro será 0+0i
zoom ← 0
centro ← 0 + 0i
Crea un array de 100 colores y Centros[zoom] ← centro
w ← 4
h ← (ancho * alto) / ancho;

// ***** A REPETIR CADA VEZ QUE DIBUJES
// Parte real de los puntos complejos donde comienzas y acabas de dibujar
xmin ← centro[zoom] - w/2;
xmax ← xmin + w;
// Parte imaginaria de los puntos complejos donde comienzas y acabas de dibujar
ymin ← centros[zoom].imaginario - h/2;
ymax ← ymin + h;
// Maximo número de iteraciones en cada punto del plano
maxIter ← 100;
// Calcular que punto complejo representa cada píxel, o visto de otra forma, cuando pasas del
// pixel actual al de al lado, a qué punto del plano cambias, cuanto incrementas (depende del
// zoom) a los que llamamos distancia x (dx) y distancia y (dy)
dx = (xmax - xmin) / ancho;
dy = (ymax - ymin) / alto;
// recorrer los pixels (que se corresponden con un punto del plano complejo)
y ← ymin;
PARA j = 0 HASTA altura HACER
    x ← xmin;
    PARA i = 0 HASTA ancho-1 HACER
        // Iterar z = z^2 + c a ver si tiende al ininito
        a ← x;       b ← y;      int n ← 0;
        max ← 4.0;  // Infinito en nuestro caso va a ser 4, cuando alcance o supere 4, se va.
        absAnt = 0.0;
        converge = maxIter;
        MIENTRAS n < maxIter HACER
            // z = a + bi
            aa ← a * a;
            bb ← b * b;
            abs ← raíz cuadrada de (aa + bb);
            SI abs > max ENTONCES
                distanciaAlUltimo ← abs - absAnt;
                distanciaAlMax  ← max - absAnt;
                converge ← n + distanciaAlMaximo / distanciaAlUltimo;
                break;
            FINSI
            abDoble ← 2.0 * a * b;
            a ← aa - bb + x;    // Es una forma de hacer z --> z^2 + c donde z=a+bi y c=(x,y)
            b ← abDoble + y;
            n++;
            absAnt = abs;
        FINMIENTRAS
        // Dar color al pixel según cuantos pasos le ha costado llegar al infinito
        // Si no ha llegado, se queda de color negro, cuanto más haya tardado, color más claro
        SI n == maxIter
        ENTONCES Pon de color negro el pixel (i,j)
        SINO
```

```
            SI colores[converge] no está definido  ENTONCES
                colores[converge] ← aleatorio (más claro cuanto mayor es converge)
            FINSI
            Colorea el pixel (i,j) de color colores[converge]
        FINSI
    FINSI
    x ← x + dx;
    FINPARA
  y ← y + dy;
FINPARA
```

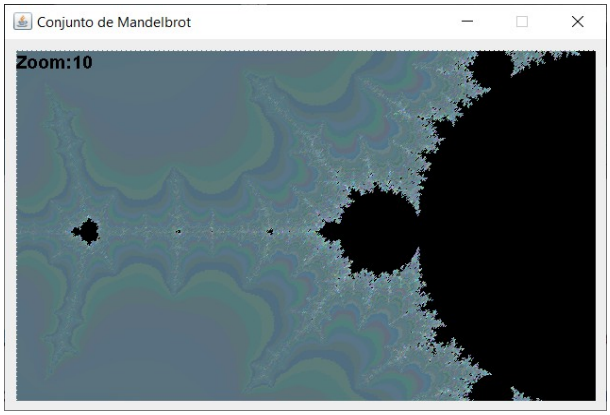
Si lo piensas detenidamente, hacer un zoom x2 para acercarte, consiste en el código en disminuir la cantidad de plano complejo que representan los pixels.

Zom x2

```
SI zoom < máximo nº zooms que puedes hacer
ENTONCES
    centro ← punto que corresponde el pixel
    zoom ← zoom + 1
    centros[zoom] ← centro
    w ← w/2
    Repinta el gráfico
FINSI
```

Zoom /2 (deshaces el zoom)

```
SI zoom >= 0
ENTONCES
    zoom ← zoom - 1
    centro ← centros[zoom]
    w ← 2 * w
    Repinta el gráfico
FINSI
```



EJERCICIO 9 (1 PUNTO). JUEGO DE SUBMARINO DE LA UNIDAD.

Modifica el código del juego, para que antes de jugar una partida, aparezca un diálogo con los nombres y puntuaciones de los 5 mejores jugadores (guardados en un fichero) y un botón [Jugar] o [Ok] (si quieres otro para salir del programa, pero es opcional) para iniciar la partida.

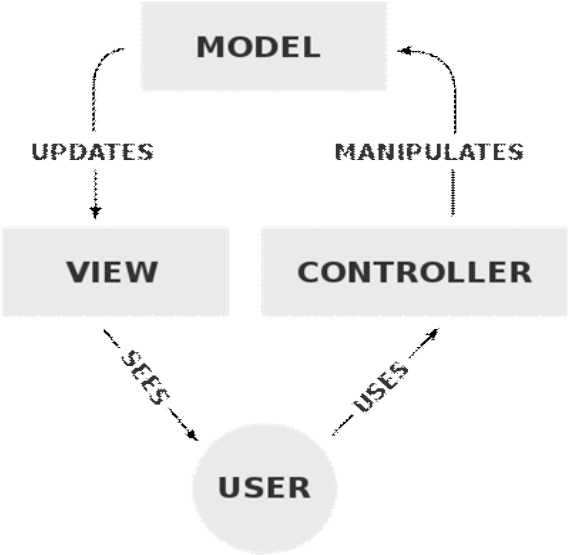
En cada partida, cada disparo fallido puntúa -0.25, cada impacto puntúa +1. Al quinto impacto, la partida acaba. Debes ir mostrando por pantalla los puntos que tiene el jugador.

La partida acaba cuando el jugador pulsa la tecla escape (ESC) o se hunde el submarino, y vuelve a aparecer el diálogo con las puntuaciones. Comprueba si el jugador debe aparecer en el top 5, y si es así, pregunta por su nombre y crea una entrada en el fichero.

EJERCICIO 10 (1 PUNTO). PATRÓN DE DISEÑO MVC.

Un patrón de diseño o caso de uso, es una forma de organizar el código de un programa (un patrón arquitectónico) que se ha demostrado útil para resolver cierto problema o característica que la aplicación debe cumplir. Por ejemplo, el singleton que ya comentamos en la unidad de POO es uno de ellos y hay muchos más: DAO, etc. En este ejercicio vamos a tomar contacto mediante un ejemplo sencillo con MVC, en el que queremos dividir el código en 3 elementos independientes:

- **Modelo (M):** contiene los datos, la lógica y las reglas de nuestra aplicación.
- **Vista (V):** se usa para mostrar los datos al usuario.
- **Controlador (C):** acepta las órdenes del usuario y las convierte en órdenes para el modelo o la vista.



El modelo es por tanto una abstracción de algo que tiene sentido para el usuario de la aplicación. Swing ofrece dos tipos de modelos:

- **GUI-state models:** son interfaces que definen el estado visual de un control de la interface, por ejemplo para saber si un botón está pulsado sería un ButtonModel.
- **Application-data models:** son una interface que representa datos que muestra al usuario, como los valores que aparecen en una tabla de datos, por ejemplo y que existen en forma de TableModel.

La vista tiene componentes de interface de usuario responsables de mostrar los datos al usuario: layouts, dibujo, etc. Un componente Jtable es un buen ejemplo de vista.

Un controlador encapsula el código de la aplicación que se ejecuta cuando un usuario interacciona (movimientos del ratón, clics de ratón, pulsaciones de teclas, etc.). En swing a los controladores se implementan normalmente mediante ActionListener o mediante Action.

Vas a implementar un sencillo ejercicio que permita filtrar datos sobre acciones de

algunas compañías. La aplicación tendrá una caja de texto donde el usuario pueda teclear un string que se usa para filtrar las empresas que quiere ver, un botón para realizar el filtrado y una tabla donde aparecen los resultados.

Modelo: Se implementa en la clase llamada Modelo<<Nombre>> (Le añades tu nombre a la clase) y que implementará la interface TableModel (o, una subclase de la clase abstracta AbstractTableModel). El papel de esta clase es servir de interface entre los datos y el componente de la vista JTable. También añasimos una clase llamada Constantes<<Nombre>> donde definimos constantes que usaremos en el código. Aquí aparece el código que debes crear:

Constantes.java

```java
package mvc_ejemplo;

public class Constantes {

    public static final Object[] CABECERA_TABLA = { "Empresa", "Nombre", "Precio",
                                                    "Cambio", "% Cambio", "Volumen" };

    public static final Object[][] DATOS = {
        { "BAC", "Bank of America Corporation", 15.98, 0.14, "+0.88%", 32157250 },
        { "AAPL", "Apple Inc.", 126.57, -1.97, "-1.54%", 31367143 },
        { "ABBV", "AbbVie Inc.", 57.84, -2.43, "-4.03%", 30620258 },
        { "ECA", "Encana Corporation", 11.74, -0.53, "-4.33%", 27317436 },
        { "VALE", "Vale S.A.", 6.55, -0.33, "-4.80%", 19764400 },
        { "FB", "Facebook, Inc.", 81.53, 0.64, "+0.78%", 16909729 },
        { "NOK", "Nokia Corporation", 8.06, 0.01, "+0.12%", 13611860 },
        { "PCYC", "Pharmacyclics Inc.", 254.67, 24.19, "+10.50%", 13737834 },
        { "RAD", "Rite Aid Corporation", 7.87, -0.18, "-2.24%", 13606253 }
    };
}
```

Modelo.java:

```java
package mvc_ejemplo;

import javax.swing.table.DefaultTableModel;

@SuppressWarnings("serial")
public class Modelo extends DefaultTableModel {

    public Modelo() {
        super(Constantes.DATOS, Constantes.CABECERA_TABLA);
    }
}
```

La Vista contiene los elementos de la GUI: un JtextField donde teclear la empresa, un JButton para indicar que filtre y el JTable donde aparece el resultado:

```java
package mvc_ejemplo;

import java.awt.Dimension;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JSplitPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.border.TitledBorder;

```

```java
public class Vista {

    public Vista() {
        JTextField tfBusca = new JTextField(26);
        JButton btFiltra = new JButton("Filtra");
        JTable tabla = new JTable();
        // Crea el modelo
        Modelo modelo = new Modelo();
        tabla.setModel(modelo);
        // Crea el controlador
        Controlador control = new Controlador(tfBusca, modelo);
        btFiltra.addActionListener(control);
        // Fija el layout
        JPanel p = new JPanel();
        p.add(tfBusca);
        p.add(btFiltra);
        JScrollPane spTabla = new JScrollPane(tabla);
        spTabla.setPreferredSize(new Dimension(700, 182));
        spTabla.setBorder(
           BorderFactory.createTitledBorder(BorderFactory.createEtchedBorder(),
                                            "Mercado",
                                            TitledBorder.CENTER,
                                            TitledBorder.TOP)
                    );
        JSplitPane sp = new JSplitPane(JSplitPane.VERTICAL_SPLIT, p, spTabla);
        sp.setDividerLocation(35);
        sp.setEnabled(false);
        // Muestra todo en una ventana deslizante
        JFrame frame = new JFrame("Primer contacto con MVC");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.add(sp);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

El Controlador: implementa la interface ActionListener, se invocará como resultado de una acción del usuario sobre la vista. El Fichero se llama Controlador.java:

```java
package mvc_ejemplo;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JOptionPane;
import javax.swing.JTextField;
import javax.swing.table.DefaultTableModel;

public class Controlador implements ActionListener {
    private JTextField busca = new JTextField(26);
    private DefaultTableModel modelo;

    public Controlador(JTextField b, DefaultTableModel m) {
        super();
        this.busca= b;
        this.modelo = m;
    }

```

```java

    @Override
    public void actionPerformed(ActionEvent e) {
        String sBusca= busca.getText();
        if (sBusca != null && !"".equals(sBusca)) {
            Object[][] nuevoDato = new Object[Constantes.DATOS.length][];
            int idx = 0;
            for (Object[] o: Constantes.DATOS) {
                if( "*".equals(sBusca.trim()) ) {
                    nuevoDato[idx++] = o;
                }
                else {
                    if(String.valueOf(o[0]).startsWith(sBusca.toUpperCase().trim())) {
                        nuevoDato[idx++]= o;
                    }
                }
            }
            modelo.setDataVector(nuevoDato, Constantes.CABECERA_TABLA);
        }
        else {
            JOptionPane.showMessageDialog(null, "Cadena de Búsqueda Vacía", "Error",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Por último, nos queda pegar todos estos elementos para que trabajen juntas en otra clase:

```java
package mvc_ejemplo;

import javax.swing.SwingUtilities;

public class MVCEjemplo {

    public static void main(String[] args) {
        SwingUtilities.invokeLater( new Runnable() {
            public void run() {
                try {
                    creaYMuestraGUI();
                }
                catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    public static void creaYMuestraGUI() throws Exception {
        new Vista();
    }
}
```

Haz en un proyecto estas clases y:

- (0.5) Entrega el código.
- (0.2) Una captura de pantalla con la ejecución donde el Frame tenga el título "Buscar cotización de <Tu nombre> + <primer apellido>".
- (0.3) Responde estas preguntas:

- ¿Qué ventajas tiene usar el MVC? (Busca en Internet si no se te ocurre).
- Si en el programa dejas el string de búsqueda vacío, ¿Qué ocurre?
- ¿Qué ocurre si escribes un asterisco en la cadena de búsqueda?
- ¿Qué cadena de búsqueda hace que aparezcan estas dos empresas en la tabla?

Mercado

Empresa	Nombre	Precio	Cambio	% Cambio	Volumen
AAPL	Apple Inc.	126.57	-1.97	-1.54%	31367143
ABBV	AbbVie Inc.	57.84	-2.43	-4.03%	30620258

a) * b) A* c) A d) *A e) cadena vacía

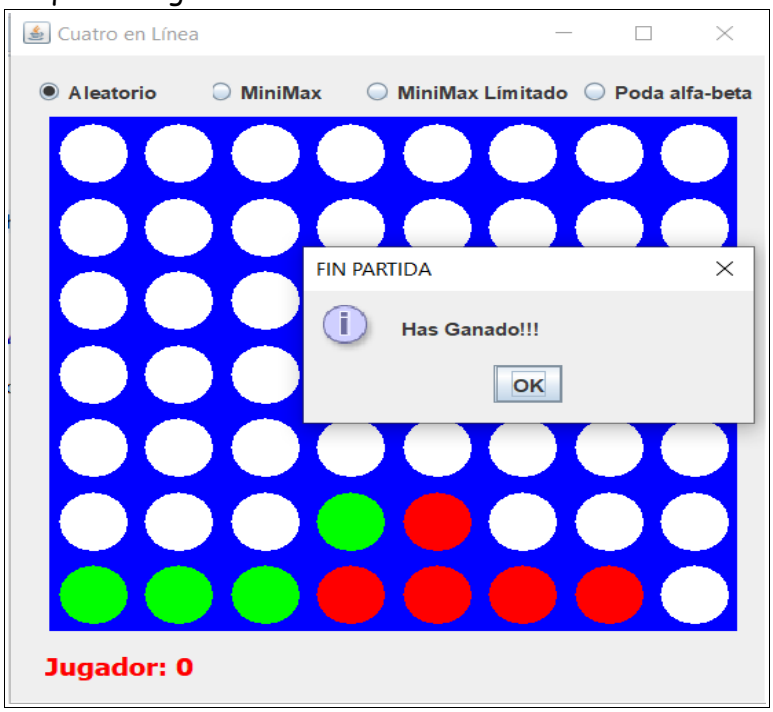
EJERCICIOS OPCIONALES (SACA ALGUNOS PUNTOS EXTRA!!)

EJERCICIO 11 (OPCIONAL +5 PUNTOS): JUEGO 5 EN LÍNEA.

Crea una versión gráfica del juego 4 en línea para jugar contra el ordenador.

(+1 PUNTO OPCIONAL) La interfaz debe permitir escoger el modo de juego (aleatorio por defecto) del programa mediante un grupo de radios: Al azar, minimax, minimax con limite de profundidad y función heurística mejorada y Poda alfa beta. Como tablero del juego te aconsejo que uses un panel de tamaño fijo para simplificar el dibujo, que se sortee el turno y se indique mostrando/ocultando un label con el color de las fichas de cada uno (Jugador/Programa) y el nº de partidas que lleva ganadas cada uno.

Cuando le toque al jugador, aparece debajo un aviso de que le toca. Cuando tenga el turno, al hacer clic en el panel, averigua en qué columna, y si tiene huecos, haz como si hubiese dejado una ficha en ella, que debe bajar hacia el fondo. Si el jugador hace clic cuando no tiene el turno, no pasará nada, se ignora.



Por definir la interfaz, dibujar el tablero, intercambiar turnos entre programa y jugador, permitir añadir fichas al jugador de esta forma y al programa de forma aleatoria: + 1 Punto.

OPCIONAL (+1 PUNTO): Si el programa es capaz de jugar una partida en modo aleatorio, es decir, verificando si gana alguno de ellos. Si escoge una opción no implementada, que de un

aviso en un diálogo y vuelva a poner la opción implementada.

OPCIONAL (+1 PUNTO): Por implementar el juego usando el método MINIMAX que permita al programa ser competitivo ("inteligente") en el juego. En este caso, la primera jugada que haga el programa que sea dejar su ficha en una columna del centro, a partir de ahí, que use minimax para decidir la columna.

OPCIONAL(+1PUNTO): Por implementar el juego usando el método MINIMAX pero intentando acelerar su respuesta. Esto implica mejorar la función heurística y limitar la búsqueda en profundidad del árbol del juego a digamos 10 niveles (puede haber 56 con 8 posibilidades en cada uno).

OPCIONAL (+1PUNTO): Por implementar el juego usando el algoritmo Poda alfa-beta (un MINIMAX mejorado).

EJERCICIO 12 (OPCIONAL): TRABAJAR CON IMÁGENES.

Haz un programa que permita seleccionar una imagen mediante diálogos accesibles desde un menú y barra de herramientas (usa acciones), cargarla en el programa para manipular sus píxels, mostrar la imagen original y la transformada en dos scrollpanels separados por un splitpanel horizontal y vertical y ofrecer al usuario mediante un menú y barra de acciones las siguientes operaciones:

- **Operación 1**: Transformarla a Escala de Grises → Una pista:

```
Color c = new Color(image.getRGB(j, i));
int red = (c.getRed() * 0.299);
int green =(c.getGreen() * 0.587);
int blue = (c.getBlue() *0.114);
```

```
int sum = red+green+blue;
Color newColor = new Color(sum,sum,sum);
image.setRGB(j,i,newColor.getRGB());
```

- **Operación 2**: Normalizar el color → cada píxel (R, G, B) a (R/ Max(R) * 255, G/Max(G) * 255, B/Max(B) * 255)

- **Operación 3**: Fundir otra imagen más pequeña dentro de ella → Elegir otra imagen de menor o igual tamaño y pedir un porcentaje de visualización, de forma que se mezclen los pixels de la nueva imagen con los de la anterior de la siguiente forma:

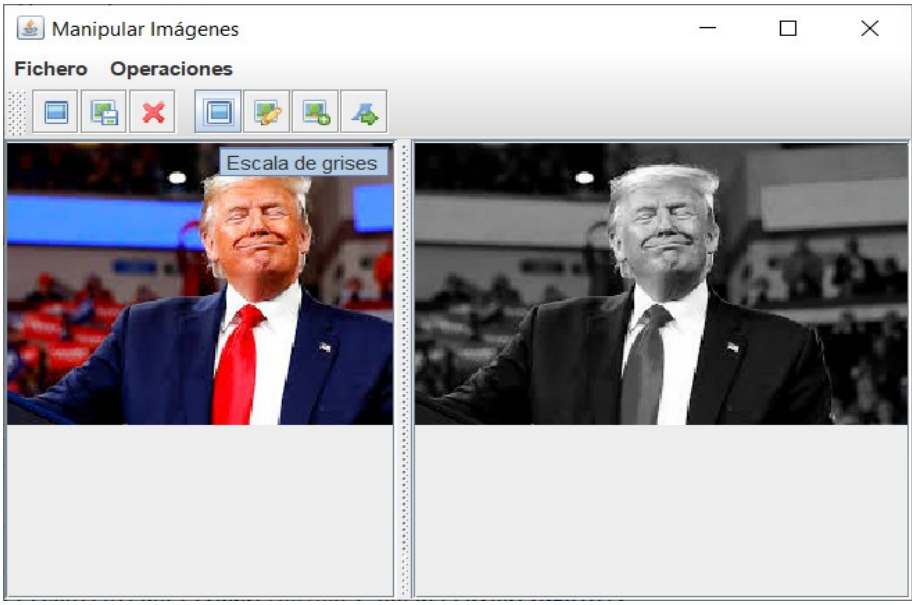
  p2 ← nº entre 0 y 1 (% de peso de la nueva imagen) indicado por el usuario
  p1 ← 1 – p1 entre 0 y 1 (% de peso de la imagen que sirve de fondo).
  Cada píxel de la imagen de base (R1, G1, B1) se mezcla con cada pixel de la nueva imagen (R2, G2, B2) donde haya solapamiento claro, de la forma:

  (R1 * p1 + R2 * p2, G1 * p1 + G2 * p2, B1 * p1 + B2 * G2)

- **Operación 4**: Exportar el resultado/Imagen cargada a un fichero de texto ASCII. Debes hacer que cada píxel genere un carácter. Para cada píxel de la imagen, averiguas

su brillo (la media de los componentes) y usando estos caracteres que van del más oscuro al más claro ['@', '#', '%', '&', '$', '+', '*', ';', ':', ',', '.', '`', ' '] Así que divides los posibles brillos [0-255] en tantos intervalos como letras uses y asocias cada brillo de un píxel a su letra.

- **Operación 5**: Guardar el resultado a una nueva imagen. Debe dejarte elegir el formato de salida de la nueva imagen (.jpg, .png, .gif, etc.).



EJERCICIO 13 (OPCIONAL +2 PUNTOS). ETAPA 4 DEL ARCADE.

ETAPA 4. LOS TOQUES FINALES.

Crea un nuevo package llamado etapa4 y copia todos los ficheros fuente y recursos de la etapa3 a ese nuevo package (es una forma cutre de tener versiones, pero es sencilla).

PASO 4.1. CONTEAR LA PUNTUACIÓN.

El paso es trivial. Cuando matamos un alien, aumenta la puntuación del jugador en 20 puntos. Este cambio lo haremos en el método colision() de la clase Alien. La puntuación es un atributo del jugador y la clase Alien no tiene acceso a esta variable. Dado que es necesario acceder a la instancia del jugador desde muchos sitios, por ejemplo para aumentar el número de bombas, para disminuir la vida, etc., añadimos un método a la interface Escenario para obtener la instancia del jugador desde cualquier punto del programa:

```
public Jugador getJugador();
```

Y lo implementamos en Bichos:

```
public Jugador getJugador() { return jugador; }
```

Ahora que podemos obtener una referencia al jugador, utilizamos este método para

```
// Crea el método si no está definido en Jugador
escena.getJugador().sumaPuntos(20);
```

aumentar la puntuación cada vez que un alien es destruido. Desde la clase Alien, en el método colision(), si colisiona con un misil o una bomba aumenta la puntuación.

PASO 4.2: CUIDADO!! NO SOMOS INMORTALES.

Un juego en el que nunca podemos perder no tiene ninguna gracia, así que es hora de afrontar este doloroso momento. Haremos que cada vez que un alien choca con el jugador, el nivel de vida disminuya y si llega a 0 el juego termina. Implementa un indicador de "juego finalizado" en la clase principal. Cualquiera puede activar el indicador llamando al método **setGameOver()**. EL bucle del método juego() ahora se modifica para que pueda acabar de esta forma:

```
private boolean acaba = false;

public void setGameOver() { acaba = true; }
```

```
while( isVisible() && !acaba) { ...
```

Crea el método **pintaGameOver(Graphics2D)** y lo llamas cuando proceda:

```
pintaGameOver() { ... ;
```

```
public void colision(Personaje p) {
    if( p instanceof Alien ) {
        p.borra();
        sumaPuntos(40);
        vida-= 40;
        // quizás debas cambiar la interface
        if( vida < 0 )
            escena.setGameOver();
    }
}
```

Cuando el juego termina, simplemente pintamos "GAME OVER" sobre la pantalla y nos quedamos esperando a que el usuario cierre la ventana. Por supuesto, en un juego "pro" haríamos alguna otra cosa: ofrecer reiniciar el juego, volver a algún menú inicial, apuntar la puntuación conseguida, etc. Veamos ahora los cambios en la clase Jugador, añadimos su propia gestión de colisión.

PASO 4.3. LOS ALIENS DISPARAN.

Hasta este punto nuestro juego parecía más un juego de tiro al pato. Ahora añadimos la posibilidad de que los aliens nos paguen con la misma moneda. Define un nuevo personaje llamado "Laser", que representa un disparo de un alien. Utilizaremos la siguiente secuencia de gráficos: "disparo0.gif", "disparo1.gif" y "disparo2.gif". La clase Laser es muy similar a la clase Misil. Las únicas diferencias son el hecho de que el Laser tiene varios fotogramas (y por tanto una velocidad de fotogramas), que se desplaza hacia abajo y que es más lento que el misil, dale una velocidad de 3 (define una constante).

```
package etapa4;

public class Laser extends Personaje {
    protected static final int LASER_VELOCIDAD = 3;

    public Laser(Escenario e) {
        super(e);
        setSpriteNombres(
                new String[] {"disparo0.gif",
                              "disparo1.gif",
                              "disparo2.gif"}
                        );
        setFrameVelocidad(10);
    }

    public void actua() {
        super.actua();
        y+= LASER_VELOCIDAD;
        if( y > Escenario.LIMITE ) borra();
    }
}
```

Los aliens disparan al azar. En cada turno, un alien tiene una probabilidad del 1% de disparar. Considerando que hay casi un centenar de turnos por segundo, esta probabilidad proporciona un ritmo de disparo bastante aceptable. De hecho, cualquier otra frecuencia de disparo hace que nos enfrentemos a una lluvia de láser. El proceso de disparo es muy similar al del jugador: se crea un nuevo actor (Laser) y se coloca justo debajo del alien que lo ha disparado. Lo lógico es que modifiques el método actua() de Alien.

```
// En la clase Alien
protected static final double FRECUENCIA_DISPARO = 0.01;
// En actua
if( Math.random() < FRECUENCIA_DISPARO ) dispara();

public void dispara() {
    Laser m = new Laser(escena);
    m.setX( x + getAncho()/2);
    m.setY( y + getAlto() );
    escena.insertaPersonaje(m);
}
```

Finalmente debemos hacer que el jugador reaccione a las colisiones con los disparos. Por cada colisión, disminuiremos la vida en 10.

```
if( p instanceof Laser ) { p.borra(); vida-= 10; }
```

PASO 4.4. UN FONDO CON SCROLL

Muchos juegos ofrecen un fondo con scroll sobre el que se desarrolla la acción. Este efecto es muy fácil de conseguir si simplemente en lugar de borrar la pantalla con un color sólido la rellenamos con una textura. El scroll se consigue modificando el rectángulo que actúa como ancla de la textura. Para entender el proceso, imagina que tenemos una imagen (256x256) que utilizamos como textura del fondo. Si rellenamos toda la pantalla con esta textura utilizando un rectángulo ancla de (0, 0, 256, 256), obtenemos un resultado. Si utilizamos un rectángulo ancla de (0, 20, 256, 256), es decir, si aumentamos la coordenada Y del rectángulo en 20 unidades, obtenemos otro efecto.

La única clase que cambia en este paso es la clase Bichos en la que importamos y añadimos variables miembro. Y algunos cambios en pintaMundo().

```
import java.awt.TexturePaint;

private BufferedImage fondo;
private int t;
```

El *efecto* es como si todo el decorado se desplazase hacia abajo 20 unidades y así conseguiremos nuestro efecto. Como queremos un desplazamiento suave, aumentamos la coordenada en un pixel cada vez. También utilizaremos una imagen más apropiada para un fondo, como por ejemplo una que **te vas ha hacer tu mismo usando el paint por ejemplo, crea una imagen de 256x256, dibuja un fondo negro y saltea algunos puntos de color blanco,**

```
public void pintaMundo() {
    Graphics2D g = (Graphics2D)strategy.getDrawGraphics();
    fondo = sc.getSprite("fondo.gif");
    g.setPaint(
      new TexturePaint(fondo,
          new Rectangle( 0, t,
                         fondo.getWidth(),
                         fondo.getHeight()
                       )
                     )
    );
    g.fillRect(0, 0, getWidth(), getHeight() );
    for (int i = 0; i < personajes.size(); i++) { ...
```

amarillo y azul simulando estrellas. Guárdala como "fondo.gif".

Por último en el método juego() cambia lo destacado :

```
public void juego() {
    fps = 1000;
    t = 0;
    iniciaMundo();
    while ( isVisible() && !acaba ) {
        t++;
```

PASO 4.4: SONIDOS.

Añadir un sonido al juego es relativamente fácil y tiene un impacto radical sobre el aspecto final del juego. En realidad, muchos de los juegos que se producen en la actualidad invierten mucho más dinero en gráficos y sonido que en programación y diseño, con la consecuencia de que a veces se obtienen juegos con unos excelentes efectos visuales y bandas sonoras pero con escasa jugabilidad. Utilizamos archivos .wav para distintas situaciones (disparos, impacto contra un alien, etc.) así como un .wav como "música de ambiente":

Archivo	Uso
musica.wav	Música ambiente
foton.wav	Alien disparando
explosion.wav	Alien destruido
misil.wav	Jugador disparando

Los sonidos son un recurso más para nuestro programa Java, de forma que lo que necesitamos es una clase que gestione una "caché de sonidos" parecida a la caché de sprites que ya tenemos. Dado que ambas comparten gran parte de la funcionalidad, podemos intentar reorganizar el código extrayendo una clase común, a la que llamaremos **RecursosCache**. Su código será el siguiente:

```
package etapa4;

import java.net.URL;
import java.util.HashMap;
import java.io.File;

public abstract class RecursosCache {
    protected HashMap recursos;

    public RecursosCache() { recursos = new HashMap(); }

    protected Object cargaRecurso(String nombre) {
        URL url= null;
        url = getClass().getClassLoader().getRecurso("etapa4/res/");
        return cargaRecurso( new File(url.getPath() + nombre) );
    }

    protected Object getRecurso(String nombre) {
        Object res = recursos.get(nombre);
        if(res == null) {
            res = cargaRecurso( nombre );
            recursos.put(nombre, res);
        }
        return res;
    }

    protected abstract Object cargaRecurso(File f);
}
```

Modificamos SpriteCache:

```
package etapa4;

import java.awt.image.BufferedImage;
import java.net.URL;
import javax.imageio.ImageIO;
import java.io.File;


public class SpriteCache extends Recursoscache implements ImageObserver {

    protected Object cargaRecurso(File f) {
        try {
          return ImageIO.read(f);
        } catch (Exception e) {
          System.out.println("Error al cargar imagen " + f);
          System.out.println("ERROR: " + e.getClass().getName()
                             + " " + e.getMessage());
          System.exit(0);
          return null;
        }
    }

    public BufferedImage getSprite(String nombre) {
        return (BufferedImage)getRecurso(nombre);
    }
}
```

Y finalmente añadimos una nueva clase **SonidoCache** que será la encargada de localizar y cargar los archivos de sonido. Su código es muy similar a SpriteCache, pero devuelve AudioClips en lugar de BufferedImages. También proporciona dos métodos adicionales que son **tocaSonido()** para reproducir un sonido una única vez y **bucleSonido()** para reproducir un sonido cíclicamente.

```
package etapa4;

import java.applet.Applet;
import java.applet.AudioClip;
import java.net.URL;
import java.io.File;

public class SonidoCache extends RecursosCache {

    protected Object cargaRecurso(File f) {
        try{ return Applet.newAudioClip( f.toURL() ); } catch(Exception e) {};
        return null;
    }

    public AudioClip getAudioClip(String nombre) { return (AudioClip)getResource(nombre);}

    public void tocaSonido(final String nombre) { getAudioClip(nombre).play(); }

    public void bucleSonido(final String nombre) { getAudioClip(nombre).loop(); }
}
```

Puesto que cualquier clase puede solicitar en un momento dado que se toque un sonido, cualquier clase debe ser capaz de obtener una referencia al caché de sonidos. Como siempre, este tipo de métodos globales los añadimos a la interfaz Escenario.

```
public SonidoCache getSonidoCache();
```

En la clase principal, debemos construir el caché de sonidos, implementar el método getSoundCache() y hacer que se inicie la música ambiente nada más comenzar el juego:

```
private SonidoCache sonic;
```

En el constructor:

```
sonic = new SonidoCache();
```

Al acabar iniciaMundo() en juego():

```
sonic.bucleSonido("musica.wav");
```

Añade el método:

```
public SonidoCache getSonidoCache() { return sonic; }
```

Por último, debemos modificar las clases Jugador y Alien de forma que en las situaciones apropiadas (disparo, impacto, etc.) suene el sonido adecuado. Realmente en esta ocasión los cambios son triviales ya que lo único que necesitamos es insertar una llamada al método **tocaSonido()** con el sonido que necesitemos:

Al disparar un misil:

```
escena.getSonidoCache().tocaSonido("misil.wav");
```

Al detectar una colisión de un alien con un misil o una bomba:

```
escena.getSonidoCache().tocaSonido("explosion.wav");
```

O cuando el alien nos dispara:

```
escena.getSonidoCache().tocaSonido("foton.wav");
```

PASO 4.5. MEJORAR EL SONIDO USANDO VARIOS HILOS.

Si pruebas el programa, habrás observado dos cosas: en primer lugar el fps puede bajar y lo que es más grave, el juego puede funcionar "a trompicones". Otro problema es que el sonido en bucle puede no funcionar. Java necesita preparar un sonido para su reproducción y este proceso lleva tiempo. Cada vez que solicitamos que se toque algo, como lo estamos haciendo desde nuestro bucle principal, el programa se detiene mientras Java prepara el sonido. Un planteamiento mejor es crear un nuevo hilo temporal y que desde este nuevo hilo se solicite la ejecución del sonido. De esta forma el programa principal puede continuar sin que tengamos que esperar. Afortunadamente, tenemos todas las peticiones de reproducción de sonidos centralizadas en la clase SonidoCache de forma que será fácil añadir el cambio sin afectar al resto del programa.

Modifica la clase RecursoCache y añade el método:

```
protected File getFicheroRecurso(String nombre) {
    URL url= null;
    url = getClass().getClassLoader().getResource("etapa4/res/");
    return new File(url.getPath() + nombre);
}
```

Y modifica la clase SonidoCache:

```
import java.applet.Applet;
import java.applet.AudioClip;
import java.net.URL;
import java.io.File;

import javax.sound.sampled.AudioSystem;
import javax.sound.sampled.Clip;

public void tocaSonido(final String nombre) {
    new Thread(
                new Runnable() {
                    public void run() { getAudioClip(nombre).play(); }
                }
               ).start();
}

public void bucleSonido(final String nombre) {
    new Thread(
           new Runnable() {
              public void run() {
                try {
                  Clip clip = AudioSystem.getClip();
                  clip.open( AudioSystem.getAudioInputStream(getFicheroRecurso(nombre)));
                  clip.start();
                  clip.loop(Clip.LOOP_CONTINUOUSLY);
                } catch(Exception e) {}
              }
           }
           ).start();
}
```

PASO 4.6: ULTIMAR DETALLES.

Realizaremos algunas pequeñas optimizaciones que ayudarán a incrementar los fps dañados desde que implementamos el sonido y las texturas. La primera consiste en utilizar imágenes en formato "compatible". ¿Qué significa esto? Una imagen "compatible" es una imagen en memoria que tiene las mismas características (o al menos muy similares) al modo de vídeo nativo que se está mostrando en ese momento en términos de bits por pixel, transparencia, etc. Las imágenes compatibles son mucho más rápidas de dibujar que cualquier otro formato. Ahora mismo lo que estamos haciendo es utilizar el formato que a ImageIO le apetece devolvernos. Podemos optimizar esto de la siguiente forma:

1. Leemos la imagen de disco utilizando ImageIO.
2. Creamos una imagen compatible del mismo tamaño que la imagen leída.
3. Pintamos la imagen recién cargada encima de la imagen compatible.

De esta forma obtenemos una "versión compatible" de una imagen situada en el disco. Haremos todo esto en la clase SpriteCache, que es la que centraliza el acceso a las imágenes:

```
import java.awt.GraphicsEnvironment;
import java.awt.GraphicsConfiguration;
import java.awt.Graphics;
import java.awt.Transparency;
import java.awt.Image;
import java.awt.image.ImageObserver;

public BufferedImage creaCompatible(int ancho, int alto, int transparencia) {
    GraphicsConfiguration gc = GraphicsEnvironment
                                .getLocalGraphicsEnvironment()
                                .getDefaultScreenDevice()
                                .getDefaultConfiguration();
    BufferedImage compatible = gc.createCompatibleImage(ancho, alto, transparencia);
    return compatible;
}

public BufferedImage getSprite(String nombre) {
    BufferedImage carga = (BufferedImage)getRecurso(nombre);
    BufferedImage compatible = creaCompatible(carga.getWidth(),
                                              carga.getHeight(),
                                              Transparency.BITMASK);
    Graphics g = compatible.getGraphics();
    g.drawImage(carga, 0, 0, this);
    return compatible;
}

public boolean imageUpdate(Image img, int infoflags, int x, int y, int w, int h) {
    return (infoflags & (ALLBITS | ABORT)) == 0;
}
```

La siguiente optimización consiste en librarnos del dibujo repetitivo de texturas dentro del bucle principal. En su lugar, procederemos de la siguiente forma: Nada mas arrancar el juego, creamos una imagen en memoria que tiene la misma anchura que la pantalla, pero con una altura igual a la de la pantalla más el tamaño de la textura. Rellenamos esta imagen con el fondo que queremos utilizar. Durante el juego, simplemente copiamos una sub-imagen de esta imagen en memoria a la pantalla. El efecto de scroll se consigue variando la coordenada Y de inicio de la sub-imagen que pintamos. Esto nos permite limitar el uso de TexturePaint, que siempre es muy lento, a una única vez y además fuera del bucle principal del juego.

Y otra optimización consiste en librarnos del cursor del ratón. Otra mejora trata con un problema que es posible que nos hayamos encontrado durante el juego. Si movemos la ventana a un segundo plano y la volvemos a poner en primer plano, veremos un parpadeo gris por unos instantes. No es muy problemático, pero sí poco profesional.

Lo que está ocurriendo es que el AWT está lanzando un evento de redibujo desde su propio hilo, mientras nosotros estamos haciendo otra cosa. Dado que AWT no utiliza un doble búfer, por unos instantes vemos una pantalla gris completamente borrada. Para evitarlo, debemos indicar a AWT que ignore las peticiones de redibujo que provengan del sistema operativo, ya que nosotros mismos nos estamos haciendo cargo del dibujo de la pantalla. Esto se consigue llamando al método **setIgnoreRepaint()**. En el constructor escribe lo que hay destacado de amarillo.:

```
import java.awt.Toolkit;
import java.awt.Transparency;
import java.awt.Point;
import java.awt.Cursor;

private BufferedImage loseta;
private int fondoY;

// Añade al constructor:
setIgnoreRepaint(true);
BufferedImage cursor = spriteCache.creaCompatible(10,10,Transparency.BITMASK);
Toolkit t = Toolkit.getDefaultToolkit();
Cursor c = t.createCustomCursor(cursor,new Point(5,5),"null");
setCursor(c);
```

```
public void iniciaMundo() {
    personajes = new ArrayList();
    for(int i = 0; i < 10; i++){
          Personaje p = new Personaje(this);
          p.setX( (int)(Math.random()*Stage.WIDTH) );
          p.setY( i * 20 );
          int v =  (int)(Math.random()*20 - 10);
          p.setVx( (v == 0)? 1:v );
          personajes.add(p);
    }
    jugador = new Jugador(this);
    jugador.setX( Escenario.ANCHO / 2 );
    jugador.setY( Escenario.LIMITE_JUGADOR - 2 * jugador.getAlto() );
    sonidoCache.bucleSonido("musica.wav");
    loseta = sc.getSprite("fondo.gif");
    fondo = sc.creaCompatible(
                    Escenario.ANCHO,
                    Escenario.ALTO + loseta.getHeight(),
                    Transparency.OPAQUE);
    Graphics2D g = (Graphics2D)fondo.getGraphics();
    g.setPaint( new TexturePaint( loseta,
                          new Rectangle(0, 0,
                                        loseta.getWidth(),
                                        loseta.getHeight() ) ) );
    g.fillRect(0, 0, fondo.getWidth(), fondo.getHeight() );
    fondoY = loseta.getHeight();
}
```

```
public void pintaMundo() {
    Graphics2D g = (Graphics2D)strategy.getDrawGraphics();
    g.drawImage( background,
                 0, 0,
                 Escenario.ANCHO, Escenario.ALTO,
                 0, backgroundY,
                 Escenario.ANCHO, backgroundY + Escenario.ALTO,
                 this );
    for(int i = 0; i < personajes.size(); i++) {
        Personaje p = (Personaje)personajes.get(i);
        p.paint(g);
    }
    jugador.paint(g);
    pintaEstado(g);
    strategy.show();
}
```

Finalmente, tenemos un problema con el juego y es que su velocidad varía un poco dependiendo de la velocidad del ordenador. Dado que la pausa que hacemos es siempre la misma, si el ordenador es rápido la actualización del escenario será rápida y el juego irá rápido. Sin embargo, si el ordenador es lento, consumirá mucho tiempo en la actualización del escenario y

aún así estamos esperando la misma cantidad de tiempo en el bucle principal, con lo cual el juego va más lento. Como esto no es muy agradable lo corregiremos exigiendo que cada turno dure cierta cantidad fija de tiempo. Esta cantidad dependerá del número de fotogramas que queremos alcanzar. Por ejemplo si queremos 60 fps, cada turno debe durar 1000/60 = 16,66 milisegundos. De forma que en lugar de dormir siempre la misma cantidad de tiempo (10ms), en cada turno calcularemos cuánto hemos invertido en la actualización y dibujo de la pantalla y dormiremos el tiempo que nos quede:

Muestra captura de ventana de la aplicación en ejecución **"Bichos46.png"** y crea un jar ejecutable del juego.

```
public void juego() {
    tiempo= 1000;
    iniciaMundo();
    while( isVisible() && !acaba ) {
        long tiempo1 = System.currentTimeMillis();
        fondoY--;
        if( fondoY < 0) fondoY = fondoLoseta.getHeight();
        actualizaMundo();
        compruebaColisiones();
        pintaMundo();
        tiempo = System.currentTimeMillis() - tiempo1;
        do { Thread.yield(); } while( System.currentTimeMillis() - tiempo < 17);
    }
    pintaGameOver();
}
```

EJERCICIO 14 (OPCIONAL +4 PUNTOS). JUEGO DE LA BRISCA.

Retoma el juego de la brisca de unidades anteriores y añade a cada carta del juego una imagen. Aprovecha el algoritmo que ya hiciste, para poder utilizarlo en un programa GUI. Aquí tienes dos artículos de como cargar imágenes como recursos del programa. Yo te propongo una interfaz, pero solo es una propuesta, tienes libertad para utilizar la que quieras:

Artículo 1.

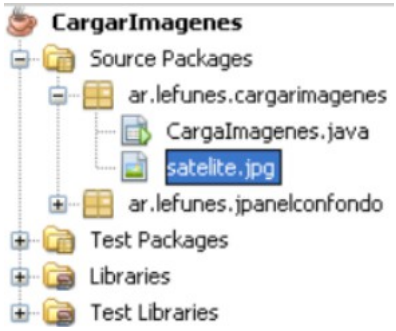
Se pueden cargar imágenes en Java, tanto si se encuentran dentro del mismo Jar de la aplicación, como si la imagen se encuentra fuera. En ambos casos vamos a utilizar la clase **ImageIcon** para recuperar la imagen. Luego, esta clase nos permite utilizarla tanto para iconos, fondos de contenedores, etc.

- **Imagen externa al Jar:** cuando la imagen se encuentra en alguna carpeta del sistema operativo. Para cargarla podemos indicarle a **ImageIcon** tanto el path relativo (desde

donde se ejecuta el Jar) como el path absoluto de la imagen. Se realiza de la siguiente forma (se utiliza **getImage()**):

```
Image imagenExterna = new ImageIcon("tierra.jpg").getImage();
```

- **Imágen interna al Jar**: cuando la imagen se encuentra dentro del Jar de la aplicación (un recurso del proyecto)
  Para cargarla podemos indicar a **ImageIcon** el recurso que se quiere utilizar. Para instanciar el recurso debemos indicar el path relativo (desde la clase que quiere cargar la imagen) como el path absoluto de la imagen dentro del Jar (un path absoluto en el jar comienza con /, como por ejemplo "/ar/lefunes/recurso/imagen.png"). Se realiza de la siguiente forma:




```
Image imagenInterna = new ImageIcon( getClass().getResource("satelite.jpg") ).getImage();
```

Artículo2

Imagina que tienes que incluir en tu proyecto de Java (de cualquier tipo) un fichero en el que tienes cierta plantilla para que el usuario de la aplicación (más bien el administrador) pueda editar la plantilla y hacer unos cambios. Si ese fichero no es un .properties clásico, lo deberás dejar dentro de un directorio de la aplicación. Por ejemplo lo podemos dejar en src/main/resources/plantilla.sql. ¿Cómo accedemos desde una clase de Java a este fichero para su lectura? La mejor manera es usar la funcionalidad para encontrar un recurso del ClassLoader de este modo:

```
URL lugar= getClass().getClassLoader().getResource("plantilla.sql");
```

Con esta instrucción busca en todo el classpath hasta encontrar el fichero. Si el fichero estuviera fuera del classpath podemos anteponer un "." de modo que sería:

```
URL lugar = getClass().getClassLoader().getResource(".plantilla.sql");
```

Una vez tenemos la localización, podemos sacar el path completo y usarlo para generar un objeto de la clase File que podamos leer con una librería de Apache FileUtils por ejemplo

```
File plantillaSQL = new File( lugar.getFile() );
String plantilla = FileUtils.readFileToString( labelFile );
```